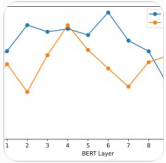
Do., 3. Sept. um 13:20



brains.sh Code	brains.sub Dokument	create_env.sh Code
create_env.sub Dokument	datalad.sh Code	datalad.sub Dokument
fmri_encoding.sh Code	fmri_encoding.sub Dokument	get_afni.sh Code
get_afni.sub Dokument	plot_layer.py Code	s41467-024-49173-... PDF

Hallo, ich soll ein Coding Projekt basierend auf dem paper machen. aktuell will ich figure 2 reproduzieren. ich habe Daten von pieman für 14 subjects... darüber haben wir schon geredet. Verschaffe dir einen Überblick über die skripte und sag mir was ich dafür machen muss.

Dauer: 1 Min. 41 Sek. >

Kurz gesagt

Dein aktueller Plot reproduziert Figure 2 noch nicht. Er zeigt die mittlere Encoding-Leistung einzelner BERT-Layer für Activations und Z-representations. Das ist eine layerweise Analyse und gehört eher in Richtung Figure 3.

Für Figure 2 musst du dagegen:

1. die Features jeweils über alle 12 BERT-Layer konkatenieren,
2. Encoding-Modelle für mehrere Featureklassen rechnen,
3. eine ISC-basierte Noise Ceiling für deine 14 Pieman-Subjects bestimmen,
4. die Ergebnisse auf die zehn linken Sprach-ROIs reduzieren,
5. pro ROI den Median über Subjects und ein 95%-Bootstrap-Konfidenzintervall darstellen.

Das Paper und das Repository bestätigen genau diesen Aufbau. Die Originalpipeline besteht dafür insbesondere aus `banded_ridge_regression.py`, `compute_isc.py`, `run_encoding_models_banded.py` und `Results_Figures.ipynb`. Repositoryübersicht,

Original-Encoding-Skript

Was Figure 2 tatsächlich zeigt

Figure 2 vergleicht fünf Featuretypen:

Modell in Figure 2	Benötigte Pieman-Features
Linguistic features	POS- und Dependency-Indikatoren
GloVe embeddings	statische, nicht kontextualisierte Wortvektoren
BERT transformation magnitudes	L2-Norm jeder Head-Transformation
BERT embeddings	Activations aller 12 Layer
BERT transformations	Z-representations aller 12 Layer

Für die beiden letzten Bedingungen werden **nicht zwölf getrennte Kurven** gezeichnet. Stattdessen werden alle Layer zu jeweils einem großen Featurevektor verbunden:

```
Nur Text

BERT embeddings:
12 Layer × 768 Dimensionen = 9216 Features pro TR

BERT transformations:
12 Layer × 768 Dimensionen = 9216 Features pro TR

Transformation magnitudes:
12 Layer × 12 Heads = 144 Features pro TR
```

Das Paper sagt ausdrücklich, dass die Transformer-Features für Figure 2 über alle Heads und Layer konkateniert wurden.

Was du bereits hast

Deine bisherige Pipeline sieht so aus:

```
Nur Text

Pieman-fMRI
    ↓
create_fmri_dataset.py
    ↓
sub-001.nii.gz ... sub-016.nii.gz
```

```
    ↓
run_encoding_pieman.py
    ↓
banded_ridge_regression.py
    ↓
24 getrennte Runs pro Subject
    ↓
12 × activations
12 × z_representations
```

Dein `</>` fmri_encoding.sh startet diese Runs für genau deine 14 Subjects:

```
</> Nur Text

sub-001 bis sub-006
sub-008
sub-010 bis sub-016
```

Dein `</>` plot_layer.py trägt anschließend die zwölf getrennten Layer-Mittelwerte auf. Das erklärt den gezeigten Plot.

Dieser Plot ist also nicht falsch. Er beantwortet nur eine andere Frage:

> Welcher einzelne BERT-Layer sagt Pieman-fMRI am besten vorher?

Figure 2 fragt stattdessen:

> Wie gut sagt die gesamte Featureklasse – über alle BERT-Layer – die Aktivität in verschiedenen Sprachregionen vorher?

Was dir für Figure 2 noch fehlt

1. All-Layer-Features erzeugen

Aus deinen vorhandenen Dateien kannst du mindestens zwei Figure-2-Inputs direkt herstellen:

```
</> Python

import numpy as np

embeddings = np.concatenate(
    [
        np.load(
            f"pieman_bert-base-uncased_layer_{layer}_activations.npy"
        )
        for layer in range(12)
    ],
```

```
    axis=1,
)

transformations = np.concatenate(
    [
        np.load(
            f"pieman_bert-base-uncased_layer_{layer}_z_representations.npy"
        )
        for layer in range(12)
    ],
    axis=1
```

Erwartet wird ungefähr:

```
Nur Text

(number_of_TRs, 9216)
```

Wichtig: Die erste Dimension muss weiterhin genau der Zahl deiner Pieman-TRs entsprechen.

Es kann sein, dass das Transformer-Notebook diese Dateien bereits erzeugen kann. Im Originalwrapper wird dafür ein Name wie

```
Nur Text

pieman_bert-base-uncased_all_layer_activations.npy
pieman_bert-base-uncased_all_z_representations.npy
```

verwendet. Prüfe zunächst:

```
Bash

cd /nethome/phussein/brains/bert-brains

find text_processing_and_transformers -type f \
  \( -iname "*pieman*all*activations*.npy" \
  -o -iname "*pieman*all*z_rep*.npy" \
  -o -iname "*pieman*magnitude*.npy" \
  -o -iname "*pieman*glove*.npy" \
  -o -iname "*pieman*linguistic*.npy" \)
```

2. Zwei neue Encoding-Bedingungen rechnen

Dein `run_encoding_pieman.py` muss zusätzlich zu den 24 Layer-Runs zwei Figure-2-Runs enthalten:

```python
representations = [
    (
        "all_layer_activations",
        "pieman_bert-base-uncased_all_layer_activations.npy",
    ),
    (
        "all_layer_z_representations",
        "pieman_bert-base-uncased_all_layer_z_representations.npy",
    ),
]
```

Dann erhältst du beispielsweise:

```
results/
├── pieman_encoding_all_layer_activations/
│   ├── sub-001_parcelwise_results_banded_ridge.npy
│   └── ...
└── pieman_encoding_all_layer_z_representations/
    ├── sub-001_parcelwise_results_banded_ridge.npy
    └── ...
```

Das ergibt zunächst eine reduzierte Figure-2-Reproduktion mit zwei Bedingungen:

- BERT embeddings
- BERT transformations

Das ist für dein Coding-Projekt ein sinnvoller erster Meilenstein.

3. Transformation magnitudes erzeugen

Dafür ist im Originalrepo vorgesehen:

```
text_processing_and_transformers/
└── transformer-transformation-magnitudes.ipynb
```

Die Magnitudes sind nicht identisch mit deinen Z-representations. Für jeden Head wird aus seinem 64-dimensionalen Transformationsvektor die L2-Norm berechnet:

$$m_{l,h,t} = \|z_{l,h,t}\|_2$$

Über 12 Layer und 12 Heads erhältst du 144 Werte pro TR.

Danach brauchst du einen weiteren Encoding-Run:

```
Nur Text

pieman_encoding_zrep_magnitudes/
```

4. Linguistic features und GloVe

Für die vollständige Figure 2 brauchst du außerdem:

```
Nur Text

extract_linguistic_features.py
transformer-/transcript-processing notebook
```

Die linguistischen Features bestehen laut Paper aus:

- 14 POS-Kategorien,
- syntaktischen Dependency-Kategorien.

GloVe wird pro Wort bestimmt und anschließend innerhalb eines TRs gemittelt.

Falls dein Projekt primär BERT und ein neueres Transformer-Modell vergleichen soll, könntest du die erste Reproduktion zunächst ausdrücklich auf die drei Transformer-Bedingungen beschränken. Eine exakte Reproduktion aller fünf Balken erfordert aber auch GloVe und linguistic features.

Der wichtigste methodische offene Punkt: Confounds

Die Originalanalyse verwendet bei jedem Encoding-Modell zusätzliche Störvariablen:

- word count,
- phoneme count,
- phoneme identity,
- silence indicator.

Diese werden gemeinsam mit den Modellfeatures gefittet, aber bei der Testvorhersage verworfen. Sie sollen verhindern, dass BERT lediglich Wort- oder Phonemrate erklärt.

Deine angepasste Pieman-Verarbeitung scheint derzeit für Pieman keine solchen `primary_features` zu verwenden. Damit ist dein Verfahren eine sinnvolle vereinfachte Reproduktion, aber noch keine methodisch exakte Figure-2-Reproduktion.

Prüfe in `banded_ridge_regression.py`:

```bash
grep -n -A100 -B10 "'pieman' in data_dir" \
  /nethome/phussein/brains/bert-brains/analysis/banded_ridge_regression.py
```

Wenn im Pieman-Zweig nur `raw_features` und fMRI geladen werden, aber nicht

```python
phoneme_vectors
phoneme_counts
word_counts
silent
primary_features
```

dann fehlen diese Confounds tatsächlich.

5. Noise Ceiling berechnen

Figure 2 zeigt nicht die rohe Korrelation r, sondern den Anteil der Noise Ceiling:

$$\text{percent noise ceiling} = 100 \cdot \frac{r_{\text{model}}}{r_{\text{ISC}}}$$

Im Repository ist dafür vorgesehen:

```
analysis/compute_isc.py
```

Das Originalskript enthält jedoch fest eingetragene Subjects für `black` und `slumlordreach`. Du musst einen Pieman-Zweig ergänzen:

```python
elif dataset == "pieman":
    subs = [
        "sub-001", "sub-002", "sub-003", "sub-004",
        "sub-005", "sub-006", "sub-008", "sub-010",
        "sub-011", "sub-012", "sub-013", "sub-014",
        "sub-015", "sub-016",
    ]
```

Danach wird für jedes Subject und jeden der 1000 Parcels dessen fMRI-Zeitreihe mit der über

Subjects gemittelten Zeitreihe korreliert. Das Paper schließt das jeweilige Subject dabei absichtlich **nicht** aus dem Gruppenmittel aus. Dadurch wird die Ceiling etwas höher und die normalisierte Modellleistung konservativer.

Erwartete Ausgaben:

```
Nur Text

pieman_data/isc/sub-001.npy
...
pieman_data/isc/sub-016.npy
```

Jede Datei sollte haben:

```
Nur Text

(1000,)
```

Wichtig ist, dass die ISC-Splits und deine Encoding-Splits exakt dieselben zeitlichen Grenzen verwenden.

6. Ergebnisse auf ROIs reduzieren

Figure 2 verwendet zehn linke ROIs:

```
Nur Text

HG
PostTemp
AntTemp
AngG
IFG
IFGorb
MFG
vmPFC
dmPFC
PMC
```

Du brauchst daher für jeden der 1000 Schaefer-Parcels eine ROI-Zuordnung. Dafür sind im Repo relevant:

```
Nur Text
```

```
data_handling/ROIs.ipynb
visualizations/Results_Figures.ipynb
```

Die Auswertung sollte konzeptionell so laufen:

```python
model_fraction = model_correlation / isc
model_percent = 100 * model_fraction
```

Dann pro Subject und ROI:

```python
subject_roi_score = np.nanmedian(
    model_percent[roi_parcel_indices]
)
```

Anschließend pro Modell und ROI:

- Marker: Median über 14 Subjects
- Fehlerbalken: 2.5- und 97.5-Perzentil einer Subject-Bootstrap-Verteilung

Negative oder sehr kleine ISC-Werte solltest du nicht einfach dividieren. Verwende beispielsweise nur Parcels mit:

```python
isc > 0
```

und dokumentiere das Auswahlkriterium.

7. Figure-2-Plot erzeugen

Die endgültige Datenstruktur sollte ungefähr so aussehen:

```
score[model, subject, roi]
```

also etwa:

```
```

```
score.shape == (5, 14, 10)
```

Für deine zunächst reduzierte Reproduktion:

```
Nur Text

score.shape == (2, 14, 10)
```

Der Plot hat dann:

- x-Achse: zehn ROIs,
- y-Achse: `% noise ceiling`,
- eine Linie/Farbe pro Featuretyp,
- Marker: Median über Subjects,
- Fehlerbalken: 95%-Bootstrap-CI.

Empfohlene Reihenfolge

Ich würde jetzt genau in dieser Reihenfolge weitermachen:

1. Prüfen, ob All-Layer-Dateien bereits existieren.
2. Falls nicht: zwölf Layer entlang `axis=1` konkatenieren.
3. Für alle 14 Subjects zwei neue Encoding-Runs rechnen:
   - all-layer activations,
   - all-layer z-representations.
4. `compute_isc.py` auf Pieman und deine Subjectliste anpassen.
5. ISC für alle 14 Subjects rechnen.
6. ROI-Zuordnung prüfen.
7. Zunächst eine reduzierte Figure 2 mit BERT embeddings und BERT transformations plotten.
8. Danach transformation magnitudes ergänzen.
9. Abschließend GloVe und linguistic features ergänzen.
10. Falls eine möglichst genaue Reproduktion verlangt wird: Pieman-Confounds in `banded_ridge_regression.py` ergänzen.

Was deine angehängten Dateien jeweils tun

Datei	Rolle
brains.sh	Erstellt Pieman-fMRI-Datensatz; Feature-Notebooks sind auskommentiert
fmri_encoding.sh	Startet deinen Wrapper für alle 14 Subjects
fmri_encoding.sub	Condor-Job für die Encoding-Runs
create_env.sh	Erstellt die fMRI-Umgebung
datalad.sh	Erstellt eine Umgebung zum Herunterladen der Narratives-Daten
get_afni.sh	Holt die Pieman-AFNI-Dateien
plot_layer.py	Zeichnet den aktuellen Layervergleich; nicht Figure 2

Die zentrale Schlussfolgerung ist daher:

> Deine Datenaufbereitung und die 14-Subject-Encoding-Pipeline stehen bereits. Für Figure 2 musst du nicht die bisherigen 24 Ergebnisse anders plotten, sondern neue All-Layer-Encoding-Modelle rechnen und anschließend durch die ISC-Noise-Ceiling normalisieren und nach linken Sprach-ROIs gruppieren.

fmri_encoding.sh
Code

plot_layer.py
Code

brains.sh
Code

3 weitere

Eingefügter Text.txt
Dokument

ok, hier noch zusatzinfos von zeug, das ich auf dem cluster erzeugt habe.

sag mir, aus welchen teilen die pipeline des papers besteht, und

welche teile davon ich aufgrund welcher daten erzeugt habe.

Dauer: 20 Sek. >

Jetzt lässt sich ziemlich genau rekonstruieren, welche Teile der Paper-Pipeline du bereits durchgeführt hast. Der aktuelle Stand ist:

> Du hast die Pieman-fMRI-Daten für 14 Subjects aufbereitet, die Pieman-Transkripte verarbeitet, BERT-Embeddings und BERT-Transformationen für alle zwölf Layer extrahiert und damit layerweise Encoding-Modelle gerechnet.
> Noch nicht vollständig vorhanden sind die eigentlichen Figure-2-All-Layer-Modelle, die Transformation Magnitudes, linguistische Features, die ISC-Noise-Ceiling und die ROI-basierte Figure-2-Auswertung.

Gesamte Pipeline des Papers

Die Pipeline kann man in sieben Teile zerlegen:

Zuordnung zu deinen Dateien

Paper-Schritt	Eingangsdaten	Ergebnis im Paper	Dein Stand
1. fMRI-Daten beschaffen	Narratives/AFNI Pieman BOLD	vorverarbeitete fMRI-Zeitreihen	vorhanden
2. Parzellierung	fMRI + Schaefer-1000-Atlas	1000 Parcel-Zeitreihen pro Subject	wahrscheinlich im Encoding-Skript durchgeführt
3. Transkript auf TRs ausrichten	Pieman-Transkript und Wortzeitpunkte	Wörter, Phoneme und Counts pro TR	zumindest für Transformer-Features durchgeführt
4. Features erzeugen	TR-aligned Text	linguistic, GloVe, BERT embeddings, transformations, magnitudes	teilweise vorhanden
5. Encoding-Modelle	Features + fMRI + Confounds	vorhergesagte fMRI-Zeitreihen und Korrelationen	layerweise vorhanden
6. Noise Ceiling	fMRI aller 14 Subjects	ISC pro Subject und Parcel	fehlt
7. Figure-2-Auswertung	Encoding-Scores + ISC + ROI-Labels	`% noise ceiling` pro Modell und ROI	fehlt

1. Beschaffung der Pieman-fMRI-Daten

Paper

Das Paper verwendet vorverarbeitete fMRI-Aufnahmen aus dem Narratives-Datensatz.

Was du gemacht hast

Du hast zunächst mit `datalad` die Pieman-Dateien aus

```
Nur Text
derivatives/afni-nosmooth/
```

geholt. Dafür sind deine Skripte zuständig:

```
datalad.sh
get_afni.sh
```

`get_afni.sh` sucht Dateien mit dem Namen:

```
task-pieman_run-1_space-MNI152NLin2009cAsym_res-native_desc-clean_bold.nii.gz
```

Anschließend hast du ausgeführt:

```bash
python data_handling/create_fmri_dataset.py pieman
```

Daraus sind die Dateien in

```
/nethome/phussein/brains/bert-brains/pieman_data/
```

entstanden:

```
sub-001.nii.gz
sub-002.nii.gz
sub-003.nii.gz
sub-004.nii.gz
sub-005.nii.gz
sub-006.nii.gz
sub-008.nii.gz
sub-010.nii.gz
sub-011.nii.gz
sub-012.nii.gz
sub-013.nii.gz
sub-014.nii.gz
sub-015.nii.gz
sub-016.nii.gz
```

Damit ist dieser Schritt für alle 14 Subjects abgeschlossen.

2. Schaefer-Parzellierung

Paper

Das Paper reduziert die fMRI-Daten auf 1000 kortikale Parcels des Schaefer-Atlas. Statt hunderttausender Voxel erhält man damit pro Subject:

```
Nur Text

1000 Parcels × Anzahl TRs
```

Was du hast

In `pieman_data/` liegt:

```
Nur Text

Schaefer1000_3mm.nii.gz
```

Damit besitzt du den benötigten Atlas.

Dein angepasstes `banded_ridge_regression.py` lädt offenbar die Subject-NIfTI-Dateien und verwendet den Atlas für die Parcel-Zeitreihen. Dass deine Ergebnisdateien parcelweise Ergebnisse enthalten, zeigt, dass dieser Schritt mindestens innerhalb der Encoding-Pipeline funktioniert:

```
Nur Text

sub-XXX_parcelwise_results_banded_ridge.npy
```

Allerdings sehe ich in `pieman_data/` noch keine separat gespeicherten Dateien wie:

```
Nur Text

sub-001_parcelwise_data.npy
```

Das ist später für das originale `compute_isc.py` relevant: Dieses Skript erwartet im Original genau solche Parcel-Zeitreihen. Für die Noise Ceiling musst du sie also entweder separat speichern oder `compute_isc.py` so ändern, dass es die NIfTI-Dateien selbst parzelliert.

Status: **für Encoding vorhanden, für ISC noch nicht vorbereitet.**

3. Verarbeitung des Pieman-Transkripts

Paper

Das Transkript wird anhand der Wortzeitpunkte auf die fMRI-TRs verteilt. Dabei werden unter anderem erzeugt:

- Wörter pro TR,
- Phoneme pro TR,
- Wortanzahl,
- Phonemanzahl,
- Silence-Indikator,
- Textfenster aus dem aktuellen und den vorherigen 20 TRs.

Diese Daten sind anschließend die Grundlage für die Transformer- und Confound-Features.

Was du gemacht hast

Du hast die Notebooks beziehungsweise Python-Versionen ausgeführt:

```
Nur Text

narratives-transcript-processing.ipynb
narratives-transcript-processing-run.ipynb
narratives-transcript-processing.py
```

Dass die Transformer-Ausgabe für Pieman existiert, zeigt, dass zumindest der für BERT notwendige TR-aligned Text erfolgreich erzeugt wurde.

Unklar ist allerdings, ob du auch alle Confound-Dateien für Pieman erzeugt hast. In deiner Dateiliste sehe ich keine Dateien wie:

```
Nur Text

pieman_word_counts.npy
pieman_phoneme_counts.npy
pieman_phoneme_vectors.npy
pieman_silent_vector.npy
pieman_nuisance_features.npy
```

Status: **Textausrichtung für Transformer abgeschlossen; Confound-Ausgabe nicht nachgewiesen.**

4. Erzeugung der Modellfeatures

Das Paper verwendet für Figure 2 fünf verschiedene Featuretypen.

4.1 BERT Embeddings

Paper

Die Embeddings sind die kontextualisierten Repräsentationen nach dem vollständigen Transformer-Layer.

Pro Layer erhält man:

```
Nur Text

Anzahl TRs × 768
```

Für Figure 2 werden alle zwölf Layer konkateniert:

```
Nur Text

Anzahl TRs × 9216
```

Was du erzeugt hast

Du hast für Pieman alle zwölf Layer erzeugt:

```
Nur Text

pieman_bert-base-uncased_layer_0_activations.npy
pieman_bert-base-uncased_layer_1_activations.npy
...
pieman_bert-base-uncased_layer_11_activations.npy
```

Diese Dateien stammen aus:

```
Nur Text

transformer-representations.ipynb
```

beziehungsweise:

```
Nur Text

transformer-representations.py
transformer-representations_executed.ipynb
```

Status:

- einzelne Layer: **vollständig vorhanden**

- Figure-2-All-Layer-Datei: **noch nicht sichtbar**

Es fehlt also noch eine Konkatenation zu beispielsweise:

```
Nur Text

pieman_bert-base-uncased_all_layer_activations.npy
```

4.2 BERT Transformations

Paper

Die Transformationen sind die Outputs des Self-Attention-Moduls. Innerhalb eines Layers bestehen sie aus:

```
Nur Text

12 Heads × 64 Dimensionen = 768 Dimensionen
```

Für Figure 2 werden wiederum alle zwölf Layer konkateniert:

```
Nur Text

12 Layer × 768 = 9216 Dimensionen
```

Was du erzeugt hast

Du hast für Pieman alle zwölf Transformationsebenen:

```
Nur Text

pieman_bert-base-uncased_layer_0_z_representations.npy
...
pieman_bert-base-uncased_layer_11_z_representations.npy
```

Status:

- einzelne Layer: **vollständig vorhanden**
- Figure-2-All-Layer-Datei: **noch nicht sichtbar**

Es fehlt eine Datei wie:

```
Nur Text

pieman_bert-base-uncased_all_z_representations.npy
```

4.3 GloVe

Paper

GloVe bildet die nicht kontextualisierte semantische Vergleichsbedingung. Die GloVe-Vektoren werden innerhalb jedes TRs über die dort gesprochenen Wörter gemittelt.

Was du erzeugt hast

Du hast:

```
Nur Text

pieman_bert-base-uncased_layer_0_glove.npy
```

Der Name ist etwas irreführend: GloVe hat keine BERT-Layer. Vermutlich speichert das Notebook die GloVe-Ausgabe beim Durchlauf von Layer 0.

Du brauchst daher nicht zwölf GloVe-Dateien. Eine einzige TR-aligned GloVe-Datei ist wahrscheinlich genau das benötigte Feature-Array.

Status: **Feature wahrscheinlich vollständig vorhanden, Encoding-Modell fehlt.**

Vor dem Run sollte einmal geprüft werden:

```
Bash

python - <<'PY'
import numpy as np

x = np.load(
    "text_processing_and_transformers/"
    "pieman_bert-base-uncased_layer_0_glove.npy",
    allow_pickle=True
)

print(x.shape, x.dtype)
PY
```

4.4 Transformation Magnitudes

Paper

Für jeden Head wird die L2-Norm seines 64-dimensionalen Transformationsvektors berechnet:

$$m_{l,h,t} = \|z_{l,h,t}\|_2$$

Das ergibt:

```
Nur Text

12 Layer × 12 Heads = 144 Features pro TR
```

Was du hast

Das dafür vorgesehene Notebook ist vorhanden:

```
Nur Text

transformer-transformation-magnitudes.ipynb
```

In der Dateiliste ist aber keine Pieman-Ausgabe wie

```
Nur Text

pieman_bert-base-uncased_zrep_magnitudes.npy
```

zu sehen.

Status: **Code vorhanden, Pieman-Featuredatei noch nicht erzeugt.**

4.5 Linguistische Features

Paper

Diese Bedingung enthält klassische linguistische Annotationen:

- POS-Tags,
- Dependency-Relationen.

Im Repository werden sie mit

```
Nur Text

extract_linguistic_features.py
```

erzeugt.

Was du hast

Das Skript ist vorhanden:

```
Nur Text

extract_linguistic_features.py
```

Es sind aber keine entsprechenden Pieman-Dateien sichtbar, etwa:

```
Nur Text

pieman_pos.npy
pieman_dependencies.npy
pieman_linguistic_features.npy
```

Status: **Code vorhanden, Pieman-Featuredateien nicht sichtbar.**

5. Zusätzliche Headwise-Distance-Daten

Du hast außerdem erzeugt:

```
Nur Text

raw_pieman_bert-base-uncased_headwise-distance_T-128_D-bck.npy
pieman_bert-base-uncased_headwise-distance_T-128_D-bck.npy
```

Diese Dateien repräsentieren die rückwärtsgerichtete Attention Distance der einzelnen Heads.

Sie gehören aber nicht zur Figure-2-Auswertung. Sie werden für spätere Analysen der funktionalen Head-Spezialisierung verwendet, insbesondere in Richtung Figure 4.

Status: **zusätzlicher späterer Paper-Schritt vorhanden, aber für Figure 2 nicht erforderlich.**

6. Banded-Ridge-Encoding

Paper

Für jede Featurebedingung wird pro Subject und pro Parcel ein Encoding-Modell gerechnet:

```
Nur Text

Features → vorhergesagte fMRI-Zeitreihe
```

Das Verfahren verwendet:

- vier zeitliche Delays,

- banded ridge regression,
- verschachtelte Cross-Validation,
- drei äußere zeitlich zusammenhängende Folds,
- Pearson-Korrelation zwischen Vorhersage und tatsächlicher fMRI-Aktivität.

Was du gemacht hast

Du hast mit

```
Nur Text

run_encoding_pieman.py
fmri_encoding.sh
```

für alle 14 Subjects Encoding-Runs gestartet.

Dabei wurden für jeden Layer zwei Bedingungen gerechnet:

```
Nur Text

layer_X_activations
layer_X_z_representations
```

Das ergibt theoretisch:

```
Nur Text

14 Subjects × 12 Layer × 2 Featuretypen
= 336 Encoding-Runs
```

Die Ergebnisordner existieren für alle Layer:

```
Nur Text

pieman_encoding_layer_0_activations
...
pieman_encoding_layer_11_activations

pieman_encoding_layer_0_z_representations
...
pieman_encoding_layer_11_z_representations
```

Die Ausgabe eines Runs besteht aus:

```
*_parcelwise_results_banded_ridge.npy
*_parcelwise_results_banded_ridge_alphas.npy
*_parcelwise_results_banded_ridge_weights.npy
```

Das bedeutet:

Datei	Bedeutung
`results_banded_ridge.npy`	Encoding-Scores pro Parcel
`alphas.npy`	gewählte Regularisierungsparameter
`weights.npy`	gelernte Regressionsgewichte

Status: **layerweise Encoding-Modelle vorhanden.**

Das sind die Ergebnisse, aus denen du deinen Plot mit Activations und Z-representations berechnet hast.

Auffälligkeit bei den Ergebnisnamen

In deinem `results/`-Listing stehen einige Dateien so:

```
pieman_encoding_layer_0_activationssub-001_parcelwise_results_banded_ridge.npy
```

Es fehlt ein `/` zwischen Ordnername und Dateiname.

Eigentlich sollte es heißen:

```
pieman_encoding_layer_0_activations/
sub-001_parcelwise_results_banded_ridge.npy
```

Das deutet darauf hin, dass bei einigen Runs `save_dir` ohne abschließenden Slash übergeben wurde:

```python
save_dir = ".../pieman_encoding_layer_0_activations"
```

während `banded_ridge_regression.py` offenbar so speichert:

```python
np.save(save_dir + sub + "...", result)
```

Dann landet die Datei neben dem Ordner statt im Ordner.

Das betrifft in deinem Listing mindestens einige `sub-001`-Ergebnisse. Die Pipeline kann trotzdem gerechnet haben, aber die Dateien liegen möglicherweise an unterschiedlichen Stellen.

Du solltest daher prüfen:

```bash
cd /nethome/phussein/brains/bert-brains

find results -type f -name \
  "sub-*_parcelwise_results_banded_ridge.npy" |
sort |
wc -l
```

Für sämtliche bisherigen Layer-Runs wären theoretisch 336 Score-Dateien zu erwarten.

Zusätzlich:

```bash
for d in results/pieman_encoding_layer_*; do
    printf "%-65s %s\n" "$d" \
        "$(find "$d" -maxdepth 1 -type f \
        -name '*_parcelwise_results_banded_ridge.npy' | wc -l)"
done
```

Pro Ergebnisordner sollten es 14 sein.

7. Was du bisher aus den Encoding-Ergebnissen berechnet hast

Du hast die Ergebnisse über Subjects und vermutlich Parcels gemittelt und daraus erzeugt:

```
activations_vs_representations.png
```

Der Plot zeigt:

```
Nur Text

x: BERT-Layer 0–11
y: mittlerer roher Encoding-Score
Linien: Activations und Z-representations
```

Damit hast du einen eigenen layerweisen Vergleich erzeugt.

Status: **gültige explorative Auswertung beziehungsweise Vorstufe von Figure 3, aber noch keine Figure 2.**

8. Noise-Ceiling-Pipeline

Paper

Das Paper berechnet für jedes Subject und jeden Parcel eine ISC-basierte Noise Ceiling:

```
Nur Text

Subject-fMRI
    korreliert mit
mittlerer fMRI-Zeitreihe aller Subjects
```

Anschließend:

$$\text{Figure-2-Wert} = 100 \cdot \frac{r_{\text{Encoding}}}{r_{\text{ISC}}}$$

Was du hast

In deiner Dateiliste sehe ich:

- keine `isc/` -Directory,
- keine `sub-XXX.npy` -ISC-Dateien,
- keine separat gespeicherten `*_parcelwise_data.npy`.

Status: **noch nicht durchgeführt.**

Das Originalskript

```
Nur Text

analysis/compute_isc.py
```

kann in deinem Setup außerdem nicht unverändert laufen, weil:

1. Pieman dort noch nicht als Dataset eingetragen ist,
2. deine 14 Subject-IDs ergänzt werden müssen,
3. es `sub-XXX_parcelwise_data.npy` erwartet, während bei dir bisher nur die NIfTI-Dateien sichtbar sind.

9. ROI-Auswertung

Paper

Figure 2 zeigt nicht einfach einen Mittelwert über alle 1000 Parcels. Die Parcel-Scores werden zehn linken Sprach-ROIs zugeordnet:

```
Nur Text

HG
PostTemp
AntTemp
AngG
IFG
IFGorb
MFG
vmPFC
dmPFC
PMC
```

Dann werden pro Modell und ROI dargestellt:

- Median über Subjects,
- 95%-Bootstrap-Konfidenzintervall.

Was du hast

Der Atlas liegt vor:

```
Nur Text

Schaefer1000_3mm.nii.gz
```

In deiner Liste ist aber keine fertige Datei sichtbar, die jeden Parcel einer der zehn Paper-ROIs zuordnet. Das entsprechende Vorgehen befindet sich wahrscheinlich in:

```
Nur Text

data_handling/ROIs.ipynb
```

```
visualizations/Results_Figures.ipynb
```

Status: **Atlas vorhanden, fertige Figure-2-ROI-Auswertung noch nicht vorhanden.**

Dein momentaner Stand als Pipeline

Statusübersicht speziell für Figure 2

Bestandteil	Status	Evidenz
14 Pieman-fMRI-Subjects	fertig	14 `.nii.gz`-Dateien
Schaefer-1000-Atlas	vorhanden	`Schaefer1000_3mm.nii.gz`
TR-aligned Pieman-Text	fertig	erfolgreiche Transformer-Extraktion
BERT embeddings pro Layer	fertig	zwölf `activations.npy`
BERT transformations pro Layer	fertig	zwölf `z_representations.npy`
GloVe	wahrscheinlich fertig	`layer_0_glove.npy`
Headwise attention distance	fertig, aber nicht Figure 2	`headwise-distance...npy`
Alle Embedding-Layer konkateniert	fehlt	keine `all_layer_activations.npy` sichtbar
Alle Transformations-Layer konkateniert	fehlt	keine `all_z_representations.npy` sichtbar
Transformation magnitudes	fehlt	nur Notebook sichtbar
Linguistic features	fehlt	nur Extraktionsskript sichtbar
Layerwise Encoding	fertig bzw. weitgehend fertig	Ergebnisordner und Scoredateien
All-Layer-Encoding für Figure 2	fehlt	keine entsprechenden Ergebnisordner
GloVe-Encoding	fehlt	kein entsprechender Ergebnisordner
ISC Noise Ceiling	fehlt	keine ISC-Dateien
linke Sprach-ROIs	noch ungeklärt	Atlas vorhanden, Zuordnung nicht sichtbar
Bootstrap-CIs	fehlt	kein Figure-2-Auswertungsskript
Figure-2-Plot	fehlt	vorhandener Plot ist layerweise

Der nächste zwingende technische Schritt ist daher nicht erneut die BERT-Featureextraktion. Deine zwölf Embedding- und Transformationsdateien sind schon da. Als Nächstes solltest du prüfen und reparieren, ob wirklich alle 336 Layer-Encoding-Ergebnisdateien sauber gespeichert wurden. Danach werden die All-Layer-Features erzeugt und die Figure-2-Encoding-Runs gestartet.

↳ pieman_bert-base-uncased_layer_0_glove.npy

wo ist das? gib mir einen befehl der mir die datei im repo anzeigt

Die Datei liegt laut deinem Listing hier:

```
Nur Text

/nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_bert-base-uncased_layer_0_
```

Existenz und Dateigröße anzeigen:

```
Bash

ls -lh /nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_bert-base-uncased_l
```

Form, Datentyp und erste Werte prüfen:

```
Bash

cd /nethome/phussein/brains/bert-brains

python - <<'PY'
import numpy as np

path = "text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy"
x = np.load(path, allow_pickle=True)

print("Datei:", path)
print("Shape:", x.shape)
print("Datentyp:", x.dtype)
print("Erster Eintrag:")
print(x[0])
PY
```

Falls du sie einfach im gesamten Repository suchen möchtest:

```bash
cd /nethome/phussein/brains/bert-brains
find . -type f -name "pieman_bert-base-uncased_layer_0_glove.npy"
```

(base) [phussein@submit text_processing_and_transformers]$ ls -lh /nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
-rw-r--r-- 1 phussein phussein 325K Aug 4 20:47 /nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy

Perfekt – die GloVe-Featuredatei für Pieman existiert und ist 325 KB groß:

```
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

Prüfe jetzt Shape, Datentyp und mögliche ungültige Werte:

```bash
cd /nethome/phussein/brains/bert-brains

python - <<'PY'
import numpy as np

path = "text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy"
x = np.load(path, allow_pickle=True)

print("Shape:", x.shape)
print("Dtype:", x.dtype)

if x.dtype != object:
    print("NaNs:", np.isnan(x).sum())
    print("Infs:", np.isinf(x).sum())
    print("Minimum:", x.min())
    print("Maximum:", x.max())

print("Erster Eintrag:", x[0])
PY
```

Für Figure 2 wäre besonders plausibel:

```
Nur Text

(277, 300)
```

also:

- 277 Pieman-TRs,
- 300 GloVe-Dimensionen.

GloVe besitzt keine Layer. `layer_0` im Dateinamen stammt lediglich aus der Struktur des Feature-Notebooks. Du verwendest daher genau diese eine Datei für den GloVe-Encoding-Run; du musst keine weiteren GloVe-Layer erzeugen.

(base) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

```
python - <<'PY'
import numpy as np

path = "text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy"
x = np.load(path, allow_pickle=True)

print("Shape:", x.shape)
print("Dtype:", x.dtype)

if x.dtype != object:
    print("NaNs:", np.isnan(x).sum())
    print("Infs:", np.isinf(x).sum())
    print("Minimum:", x.min())
    print("Maximum:", x.max())

print("Erster Eintrag:", x[0])
PY
```

```
Shape: (277, 300)
Dtype: float32
NaNs: 0
Infs: 0
Minimum: -2.9040499
```

Maximum: 3.5805
Erster Eintrag: [3.38135004e-01 2.64347568e-02
-1.79194003e-01 -3.16174701e-03
  1.78587750e-01 4.38317508e-02 1.53795257e-01
-1.26602501e-01
  4.55172509e-02 2.24707508e+00 -2.17412502e-01
2.25800276e-03
  2.09839731e-01 -2.37554759e-01 1.80591494e-01
8.67160037e-02
 -9.91897509e-02 5.64712524e-01 -6.14424944e-02
1.02499872e-03
  2.33316258e-01 -5.41342497e-02 -1.58716008e-01
1.98114991e-01
 -3.45457494e-02 5.79207465e-02 -1.64182514e-01
-1.57556012e-01
 -1.54071003e-01 2.15906993e-01 2.57100016e-02
2.75542498e-01
 -2.06282496e-01 5.05910516e-02 -2.54202485e-01
3.69742513e-02
 -5.72095066e-02 -1.55454502e-01 -1.09636500e-01
4.75105233e-02
  8.25684965e-02 -6.90112561e-02 -2.01787502e-02
9.24289972e-02
  1.10791750e-01 1.64777488e-01 -2.51309946e-02
-3.62622514e-02
  2.80352503e-01 -1.14029750e-01 -9.43657458e-02
4.60870042e-02
 -3.13534960e-02 5.85189983e-02 6.68392796e-03
-6.70400076e-03
  8.92505050e-04 -2.81574011e-01 1.89209267e-01
-1.62651494e-01
 -1.29867241e-01 -2.28150249e-01 3.38442475e-02
1.47834256e-01
 -4.43180017e-02 -1.95042491e-01 1.57305256e-01
-1.32849514e-01
  2.43399411e-01 1.84316009e-01 1.15899488e-01
-6.19892403e-02
 -2.13425010e-02 1.50412500e-01 2.81232476e-01
3.48834991e-01

1.27952501e-01 7.75917470e-02 -1.55097008e-01
6.76850006e-02
-2.09360048e-02 1.46211252e-01 -1.09186001e-01
7.18084723e-02
1.25854239e-01 -3.16858515e-02 -1.65671498e-01
-3.45377535e-01
9.28672403e-02 2.58823514e-01 -3.65869999e-01
-1.32969081e-01
-8.29659998e-02 1.90747991e-01 1.61396176e-01
1.75679997e-02
3.61640006e-01 2.81449966e-02 -1.62806749e-01
5.08707538e-02
-1.24251246e-01 -1.21760249e-01 1.02802500e-01
-1.38044246e-02
-1.64545000e-01 -7.88785040e-01 1.98479995e-01
-2.16249973e-02
1.75157502e-01 -5.05825058e-02 -7.58184046e-02
-1.90936252e-01
3.95350009e-01 -7.91622400e-02 -1.96570009e-02
-8.19442496e-02
-1.84509978e-02 -9.17350054e-02 1.28897250e-01
6.65867478e-02
2.39351243e-01 -3.52595784e-02 -9.98499990e-03
1.92896500e-01
2.45203495e-01 2.46623755e-01 -1.17490001e-01
-4.85327467e-03
1.41532533e-02 7.52568766e-02 1.92093760e-01
1.76074989e-02
-2.24496245e-01 8.96449983e-02 2.19355017e-01
-8.20605010e-02
-1.00124493e-01 2.59442776e-01 -3.35022546e-02
-8.70124921e-02
-1.88403249e+00 1.30392253e-01 7.61477500e-02
4.90920022e-02
-8.35124999e-02 4.55649942e-03 -1.52254999e-01
1.24851495e-01
-1.22978501e-01 5.18264920e-02 -1.68053240e-01
1.89475238e-01
1.17200002e-01 9.29592475e-02 -5.61890043e-02

-1.56787500e-01
-1.28602490e-01 4.69145030e-02 1.15594998e-01
-1.21900007e-01
-7.79950023e-02 -1.89077497e-01 9.17270035e-02
-9.11644921e-02
-1.60985008e-01 -1.89507753e-01 -1.26896247e-01
-5.24189994e-02
1.96252495e-01 -6.27617538e-02 -1.94990501e-01
-1.20201007e-01
4.07074988e-02 -2.63154864e-01 -1.71749264e-01
-4.96839993e-02
-1.52414247e-01 1.33197248e-01 -2.00304255e-01
2.01764982e-02
7.66072497e-02 -1.84047446e-02 -1.33655012e-01
1.49442494e-01
2.50624232e-02 -2.98350491e-02 -2.61667490e-01
-2.20583715e-02
3.92852537e-02 -3.53117511e-02 4.62299995e-02
-1.91262476e-02
-2.52615005e-01 -3.45280528e-01 8.44989941e-02
1.82613239e-01
1.98004991e-01 -2.09465012e-01 -8.88597444e-02
-1.97935000e-01
-1.43762499e-01 -1.94227509e-02 -9.73742530e-02
5.01220003e-02
1.23604998e-01 1.25607491e-01 1.26749992e-01
-3.38596165e-01
-1.23628497e-01 -1.54800504e-01 -1.04238249e-01
4.71350029e-02
-3.23267505e-02 1.04287505e-01 -1.61807001e-01
1.22777499e-01
1.31965995e-01 -3.45346749e-01 1.48725003e-01
1.71047494e-01
2.44364925e-02 -1.40964270e-01 -1.22476250e-01
1.39197499e-01
-1.94762498e-01 -3.92477512e-02 1.18994996e-01
2.18610004e-01
3.23677510e-02 2.91404966e-02 3.48249823e-03
-6.76377490e-02

-1.85817242e-01 -1.47600025e-02 -2.14584768e-01
-2.19522506e-01
-3.01274002e-01 1.59389004e-01 -1.47517487e-01
1.14397995e-01
-6.90625012e-02 6.69072494e-02 -2.82442458e-02
1.94001988e-01
-1.12987012e-01 -2.05745012e-01 1.32585004e-01
2.32040025e-02
-5.42892478e-02 2.46471003e-01 7.82922506e-02
-2.32602477e-01
-1.67624995e-01 1.30993754e-01 2.58415759e-01
-1.28349006e-01
-6.71452507e-02 1.39183000e-01 -5.18166497e-02
1.30502507e-01
3.27552497e-01 2.49095000e-02 8.35860297e-02
1.82294995e-01
-1.10918000e-01 -1.82946503e-01 -3.90372537e-02
5.87959960e-02
-3.16500008e-01 3.53002511e-02 2.27921531e-01
3.72257531e-02
-2.28464991e-01 -3.08980010e-02 2.81987488e-02
-3.90214995e-02
-1.65085003e-01 -1.49880022e-01 9.84173492e-02
1.39835000e-01
2.38379791e-01 5.93537539e-02 1.33233249e-01
-5.93092442e-02
-3.53527516e-02 -9.25890058e-02 2.61991501e-01
1.09410003e-01
-1.68632511e-02 5.82460240e-02 -1.76942497e-01
4.52985354e-02
-2.44024992e-02 1.54505000e-01 1.47212505e-01
-1.31938249e-01
2.68697739e-01 3.03902507e-01 -6.49750009e-02
-3.72891836e-02]
(base) [phussein@submit bert-brains]$

Die GloVe-Datei ist technisch vollständig und sauber:

```
Shape: (277, 300)
Dtype: float32
NaNs: 0
Infs: 0
```

Das bedeutet:

- 277 Zeilen = 277 Pieman-TRs
- 300 Spalten = 300-dimensionale GloVe-Repräsentation
- keine fehlenden oder ungültigen Werte
- bereits numerisches Format für die Regression

Du musst die Datei nicht weiter verarbeiten oder mit anderen Layern konkatenieren. GloVe ist nicht layerbasiert; `layer_0` ist hier nur Teil der Benennung durch das Notebook.

Damit ist der GloVe-Featureextraktionsschritt für Figure 2 abgeschlossen.

Der nächste Schritt ist ein Encoding-Run für diese Datei, für alle 14 Subjects. Füge dazu in deinem `run_encoding_pieman.py` eine zusätzliche Repräsentation ein:

```python
representations = [
    (
        "glove",
        "pieman_bert-base-uncased_layer_0_glove.npy",
    ),
]
```

Oder teste zunächst nur ein Subject direkt:

```bash
cd /nethome/phussein/brains/bert-brains/analysis

python banded_ridge_regression.py \
    sub-001 \
    ../text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy \
    glove \
    ../pieman_data/ \
    0 \
    300 \
    ../results/pieman_encoding_glove/
```

Vorher den Ergebnisordner anlegen:

```bash
mkdir -p /nethome/phussein/brains/bert-brains/results/pieman_encoding_glove
```

Der abschließende Slash bei

```
../results/pieman_encoding_glove/
```

ist wichtig. Ohne ihn entsteht erneut ein Dateiname wie:

```
pieman_encoding_glovesub-001_parcelwise_results_banded_ridge.npy
```

anstatt einer Datei innerhalb des Ergebnisordners.

Nach dem Test sollte Folgendes existieren:

```bash
ls -lh \
/nethome/phussein/brains/bert-brains/results/pieman_encoding_glove/
```

Erwartet werden:

```
sub-001_parcelwise_results_banded_ridge.npy
sub-001_parcelwise_results_banded_ridge_alphas.npy
sub-001_parcelwise_results_banded_ridge_weights.npy
```

Wenn dieser Test erfolgreich ist, kannst du denselben GloVe-Run über deine vorhandene `fmri_encoding.sh` für alle 14 Subjects ausführen.

wann habe ich die erzeugt? kann es sein dass sie schon im repo vorhanden war und auf unterschiedlichen daten basiert?

Dauer: 8 Sek.

Ja, das kann sehr gut sein. Meine vorherige Aussage, du hättest die Datei selbst erzeugt, war zu sicher.

Sicher wissen wir momentan nur:

```
Nur Text

Änderungszeit: 4. August, 20:47
Datei: pieman_bert-base-uncased_layer_0_glove.npy
```

Das Datum kann vom Notebook-Lauf stammen, aber ebenso vom Klonen, Kopieren oder Entpacken des Repositories.

Prüfe zuerst, ob die Datei Teil des Git-Repositories ist:

```
Bash

cd /nethome/phussein/brains/bert-brains

git ls-files --error-unmatch \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

Interpretation

Wenn der Dateipfad ausgegeben wird:

```
Nur Text

text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

dann ist die Datei von Git versioniert und war wahrscheinlich bereits im Repository vorhanden.

Wenn du bekommst:

```
Nur Text

error: pathspec ... did not match any file(s) known to git
```

dann ist sie nicht Teil des Repositories und wurde vermutlich lokal erzeugt oder hineinkopiert.

Zusätzlich den Git-Ursprung prüfen:

```bash
git log --follow --stat -- \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

Und alle Zeitstempel anzeigen:

```bash
stat \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

Außerdem kannst du prüfen, ob Git die Datei als unverändert, geändert oder untracked betrachtet:

```bash
git status --short -- \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

Ausgabe	Bedeutung
keine Ausgabe	versionierte und unveränderte Datei
`?? datei.npy`	nicht versioniert, wahrscheinlich lokal erzeugt
`M datei.npy`	versionierte Datei wurde lokal verändert
Git-Fehler bei `ls-files`	Datei gehört nicht zum Repository

Dass dieselbe Art von Datei auch für `black` und `slumlordreach` vorhanden ist, ist tatsächlich ein Hinweis darauf, dass diese Dateien bereits als vorbereitete Outputs im Repo vorhanden gewesen sein könnten. Entscheidend ist aber die Ausgabe von `git ls-files`.

(base) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

git ls-files --error-unmatch \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
error: pathspec 'text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy' did not match any file(s) known to git

Did you forget to 'git add'?
(base) [phussein@submit bert-brains]$

Damit steht fest:

> Die Datei gehört nicht zum Git-Repository. Sie wurde lokal auf deinem Cluster erzeugt oder dorthin kopiert.

Am wahrscheinlichsten wurde sie am **4. August um 20:47 Uhr** gemeinsam mit den anderen Pieman-Features durch deinen Lauf von

```
Nur Text

transformer-representations.ipynb
```

beziehungsweise der ausgeführten Version

```
Nur Text

transformer-representations_executed.ipynb
```

erzeugt.

Prüfe die Zeitstempel der zusammengehörigen Dateien:

```
Bash

ls -lh --time-style=long-iso \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_{glove,activations,z_representations}
```

Wenn alle ungefähr dieselbe Uhrzeit besitzen, wurden sie sehr wahrscheinlich im selben Notebook-Lauf gespeichert.

Du kannst außerdem nach der Codezeile suchen, welche die GloVe-Datei schreibt:

```
Bash

cd /nethome/phussein/brains/bert-brains

grep -Rni \
"glove" \
text_processing_and_transformers/transformer-representations.py \
text_processing_and_transformers/narratives-transcript-processing.py
```

Oder gezielt nach `np.save`:

```bash
grep -n -B5 -A8 \
"np.save.*glove\|glove.*np.save" \
text_processing_and_transformers/transformer-representations.py
```

Für das Notebook selbst:

```bash
jupyter nbconvert \
  --to script \
  text_processing_and_transformers/transformer-representations_executed.ipynb \
  --stdout |
grep -n -B5 -A8 "glove"
```

Falls die Datei nur erzeugt und nicht kopiert wurde, ist ihre bestmögliche lokale Erstellzeit über `stat` sichtbar:

```bash
stat \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
```

Achte auf:

```
Birth:
Modify:
Change:
```

- `Birth`: Zeitpunkt, zu dem diese konkrete Datei auf diesem Dateisystem angelegt wurde.
- `Modify`: letzte Veränderung des Dateiinhalts.
- `Change`: letzte Änderung der Metadaten.

Inhaltlich passt sie jedenfalls zu deinem Pieman-Featurelauf: `(277, 300)` entspricht genau 277 Pieman-TRs mit je einem 300-dimensionalen GloVe-Vektor. Sie basiert damit sehr wahrscheinlich auf deinem bereits TR-ausgerichteten Pieman-Transkript.

Did you forget to 'git add'?

(base) [phussein@submit bert-brains]$ ls -lh --time-style=long-iso \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_{glove,activations,z_representations}.npy
-rw-r--r-- 1 phussein phussein 832K 2026-08-04 20:47 text_processing_and_transformers/pieman_bert-base-uncased_layer_0_activations.npy
-rw-r--r-- 1 phussein phussein 325K 2026-08-04 20:47 text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
-rw-r--r-- 1 phussein phussein 832K 2026-08-04 20:47 text_processing_and_transformers/pieman_bert-base-uncased_layer_0_z_representations.npy
(base) [phussein@submit bert-brains]$

Damit ist es sehr wahrscheinlich, dass alle drei Dateien im selben Notebook-Lauf am **4. August 2026 um 20:47 Uhr** erzeugt wurden:

```
Nur Text

layer_0_activations.npy
layer_0_glove.npy
layer_0_z_representations.npy
```

Die Dateigrößen passen ebenfalls:

Datei	Shape vermutlich	Größe
Activations	`(277, 768)`	832 KB
GloVe	`(277, 300)`	325 KB
Z-representations	`(277, 768)`	832 KB

Das spricht dafür, dass `transformer-representations.ipynb` bei der Verarbeitung des Pieman-Transkripts gleichzeitig folgende Features gespeichert hat:

- BERT-Embeddings,
- BERT-Transformationen,
- GloVe-Repräsentationen.

Die Meldung

```
Nur Text

Did you forget to 'git add'?
```

ist nur eine allgemeine Git-Hinweismeldung. Sie bedeutet nicht, dass du die Datei zu Git hinzufügen sollst. Sie bestätigt lediglich, dass Git die Datei nicht verfolgt.

Damit können wir den Pipeline-Status korrigieren:

> Die GloVe-Features wurden von dir höchstwahrscheinlich am 4. August zusammen mit den Pieman-BERT-Features aus dem Pieman-Transkript erzeugt. Der Featureextraktionsschritt für GloVe ist abgeschlossen; nur der GloVe-Encoding-Run fehlt noch.

ok. was fehlt jetzt, um figre 2 zu reproduzieren (anhand des ipynb)?

Dauer: 10 Sek.

Für Figure 2 fehlen dir noch vier zentrale Verarbeitungsschritte. Deine bisherigen layerweisen Encoding-Ergebnisse kannst du dafür nicht einfach mitteln, weil Figure 2 für BERT jeweils ein neues gemeinsames Modell mit allen zwölf Layern fitten muss.

Was bereits vorhanden ist

Bestandteil	Status
Pieman-fMRI für 14 Subjects	vorhanden
Schaefer-1000-Atlas	vorhanden
BERT-Embeddings, 12 Layer	vorhanden
BERT-Transformationen, 12 Layer	vorhanden
GloVe `(277, 300)`	vorhanden
Layerweise Encoding-Modelle	vorhanden
Figure-2-All-Layer-Encoding	fehlt
Transformation Magnitudes	fehlt
Linguistic Features	vermutlich fehlt
ISC Noise Ceiling	fehlt
ROI-Auswertung und Bootstrap	fehlt

1. Die fünf Figure-2-Featuredateien vorbereiten

Figure 2 benötigt diese fünf Modellbedingungen:

Figure-2-Bedingung	Deine Eingabedatei
Linguistic features	noch zu erzeugen
GloVe	bereits vorhanden
Transformation magnitudes	noch zu erzeugen
BERT embeddings	zwölf Activations-Dateien konkatenieren
BERT transformations	zwölf Z-representation-Dateien konkatenieren

BERT-Layer konkatenieren

Erzeuge die beiden All-Layer-Dateien:

```bash
cd /nethome/phussein/brains/bert-brains

python - <<'PY'
import os
import numpy as np

feature_dir = "text_processing_and_transformers"

activations = []
transformations = []

for layer in range(12):
    a_path = os.path.join(
        feature_dir,
        f"pieman_bert-base-uncased_layer_{layer}_activations.npy"
    )
    z_path = os.path.join(
        feature_dir,
        f"pieman_bert-base-uncased_layer_{layer}_z_representations.npy"
    )
```

Erwartete Ausgabe:

```
pieman_bert-base-uncased_all_layer_activations.npy (277, 9216)
pieman_bert-base-uncased_all_layer_z_representations.npy (277, 9216)
```

Wichtig: Die zwölf alten Encoding-Scores werden nicht nachträglich zusammengefügt. Die 9216 Features werden zuerst zusammengefügt und dann in einem neuen Encoding-Modell gefittet.

2. Transformation Magnitudes erzeugen

Prüfe zunächst, ob das Notebook bereits eine geeignete Pieman-Zelle enthält:

```bash
```

```
cd /nethome/phussein/brains/bert-brains

jupyter nbconvert \
  --to script \
  text_processing_and_transformers/transformer-transformation-magnitudes.ipynb \
  --stdout |
grep -ni -B8 -A15 "pieman\|magnitude\|z_representations"
```

Falls deine Z-representations pro Layer die zwölf 64-dimensionalen Head-Outputs konkatenieren, kannst du die Magnitudes direkt erzeugen:

</> Bash

```
python - <<'PY'
import os
import numpy as np

feature_dir = "text_processing_and_transformers"
all_magnitudes = []

for layer in range(12):
    path = os.path.join(
        feature_dir,
        f"pieman_bert-base-uncased_layer_{layer}_z_representations.npy"
    )

    z = np.load(path)

    if z.shape != (277, 768):
        raise ValueError(f"Unexpected shape for layer {layer}: {z.shape}")

    # 768 = 12 Heads × 64 Dimensionen
    z_headwise = z.reshape(z.shape[0], 12, 64)
```

Erwartet:

</> Nur Text

```
(277, 144)
```

Aber: Verwende bevorzugt die Logik aus `transformer-transformation-magnitudes.ipynb`, falls sie die Magnitudes vor oder nach einer anderen Mittelungsoperation berechnet. Für eine möglichst genaue Reproduktion sollte die Reihenfolge dem Notebook entsprechen.

3. Linguistische Features prüfen

Prüfe, ob sie vielleicht schon an einer anderen Stelle liegen:

```bash
cd /nethome/phussein/brains/bert-brains

find . -type f \( \
  -iname "*pieman*pos*.npy" \
  -o -iname "*pieman*dep*.npy" \
  -o -iname "*pieman*linguistic*.npy" \
\) -print
```

Wenn keine Ausgabe kommt, musst du `extract_linguistic_features.py` für Pieman anpassen beziehungsweise ausführen.

Suche zunächst, welche Argumente das Skript erwartet:

```bash
sed -n '1,240p' \
text_processing_and_transformers/extract_linguistic_features.py
```

Die klassische Figure-2-Bedingung umfasst gemeinsam:

- POS-Indikatoren,
- Dependency-Indikatoren.

Auch diese müssen pro TR vorliegen.

4. Fünf neue Encoding-Bedingungen rechnen

Du brauchst pro Subject jeweils ein Encoding-Modell für:

```
linguistic
glove
zrep_magnitudes
all_layer_activations
all_layer_z_representations
```

Das sind:

$$14\ \text{Subjects} \times 5\ \text{Modelle} = 70\ \text{Runs}$$

Die bereits gerechneten 336 layerweisen Runs ersetzen diese 70 Runs nicht.

Für einen ersten vollständigen Figure-2-Zwischenstand kannst du zunächst drei Bedingungen rechnen:

```
Nur Text

GloVe
BERT embeddings
BERT transformations
```

Dafür sollte dein `run_encoding_pieman.py` ungefähr diese Einträge bekommen:

```
Python

representations = [
    (
        "glove",
        "pieman_bert-base-uncased_layer_0_glove.npy",
    ),
    (
        "all_layer_activations",
        "pieman_bert-base-uncased_all_layer_activations.npy",
    ),
    (
        "all_layer_z_representations",
        "pieman_bert-base-uncased_all_layer_z_representations.npy",
    ),
]
```

Später ergänzt du:

```
Python

(
    "zrep_magnitudes",
    "pieman_bert-base-uncased_zrep_magnitudes.npy",
),
(
    "linguistic",
    "NAME_DER_LINGUISTIC_DATEI.npy",
),
```

Erwartete Ergebnisordner:

```
Nur Text
```

```
results/pieman_encoding_glove/
results/pieman_encoding_zrep_magnitudes/
results/pieman_encoding_all_layer_activations/
results/pieman_encoding_all_layer_z_representations/
results/pieman_encoding_linguistic/
```

Jeder Ordner muss 14 Scoredateien enthalten.

5. ISC Noise Ceiling berechnen

Figure 2 verwendet nicht direkt die rohen Encoding-Korrelationen. Das Notebook lädt zusätzlich ISC-Werte und berechnet sinngemäß:

```python
percent_noise_ceiling = 100 * encoding_score / isc
```

Dir fehlen momentan offenbar:

```
pieman_data/isc/sub-001.npy
...
pieman_data/isc/sub-016.npy
```

Dafür musst du `analysis/compute_isc.py` auf Pieman anpassen:

- deine 14 Subjects eintragen,
- Pieman-Pfade eintragen,
- sicherstellen, dass dieselben 1000 Parcel-Zeitreihen verwendet werden,
- dieselben drei zeitlichen Folds wie beim Encoding verwenden.

Wichtig: Das Originalskript erwartet:

```
sub-XXX_parcelwise_data.npy
```

Diese Dateien sind in deinem Listing noch nicht vorhanden. Daher muss entweder:

1. die Parzellierung einmal separat gespeichert werden, oder
2. `compute_isc.py` direkt deine `.nii.gz`-Dateien plus `Schaefer1000_3mm.nii.gz` laden.

6. ROI-Zuordnung aus dem Notebook übernehmen

`Results_Figures.ipynb` muss auf deine Pfade und Pieman angepasst werden. Es benötigt die Zuordnung der 1000 Schaefer-Parcels zu den zehn linken Sprach-ROIs:

```
Nur Text

HG
PostTemp
AntTemp
AngG
IFG
IFGorb
MFG
vmPFC
dmPFC
PMC
```

Zeige dir zunächst die relevanten Notebook-Zellen an:

```bash
cd /nethome/phussein/brains/bert-brains

jupyter nbconvert \
  --to script \
  visualizations/Results_Figures.ipynb \
  --stdout > /tmp/Results_Figures.py

grep -ni -B15 -A40 \
"noise.ceiling\|isc\|HG\|PostTemp\|get_result_dataset" \
/tmp/Results_Figures.py
```

Noch übersichtlicher:

```bash
grep -nE \
"result_dir|isc|noise|roi|ROI|HG|PostTemp|AntTemp|get_result_dataset|bootstrap" \
/tmp/Results_Figures.py
```

Dort müssen anschließend insbesondere geändert werden:

- Originaldatensätze `black` und `slumlordreach` → `pieman`
- Original-Subjectlisten → deine 14 Subjects
- Original-Ergebnispfade → deine fünf Pieman-Ordner
- Original-ISC-Pfade → `pieman_data/isc/`

- gegebenenfalls Original-ROI-Dateipfade → deine lokalen ROI-Dateien

7. Median und Bootstrap-CI plotten

Figure 2 zeigt pro Modell und ROI:

- Marker: Median über Subjects,
- Fehlerbalken: 95%-Bootstrap-CI über Subjects,
- y-Achse: Prozent der Noise Ceiling.

Die fertige Auswertungsstruktur sollte ungefähr sein:

```
Nur Text

models × subjects × ROIs
5 × 14 × 10
```

Nicht:

```
Nur Text

layers × feature_types
12 × 2
```

Deine nächsten konkreten Schritte

In dieser Reihenfolge:

1. All-Layer-Activations erzeugen: `(277, 9216)`.
2. All-Layer-Z-representations erzeugen: `(277, 9216)`.
3. Transformation Magnitudes anhand des Originalnotebooks erzeugen: `(277, 144)`.
4. Linguistic-Dateien suchen beziehungsweise erzeugen.
5. 70 neue Encoding-Runs rechnen.
6. ISC Noise Ceiling für 14 Subjects × 1000 Parcels rechnen.
7. `Results_Figures.ipynb` auf Pieman, deine Subjects und Pfade anpassen.
8. Nur linke Sprach-ROIs auswählen.
9. Prozent Noise Ceiling berechnen.
10. Median und 95%-Bootstrap-CI plotten.

Der unmittelbare nächste Schritt ist das Erzeugen der beiden All-Layer-Dateien. Danach kannst du bereits GloVe, BERT embeddings und BERT transformations rechnen, während du Transformation Magnitudes, Linguistic Features und ISC vorbereitest.

welches ipynb brauche ich für figure 2, tho?

Für den fertigen Plot von Figure 2 brauchst du:

```
Nur Text

visualizations/Results_Figures.ipynb
```

Vollständiger Pfad:

```
Nur Text

/nethome/phussein/brains/bert-brains/visualizations/Results_Figures.ipynb
```

Anzeigen beziehungsweise öffnen:

```
Bash

cd /nethome/phussein/brains/bert-brains
jupyter notebook visualizations/Results_Figures.ipynb
```

Oder als Python-Code untersuchen:

```
Bash

jupyter nbconvert \
  --to script \
  visualizations/Results_Figures.ipynb \
  --stdout > /tmp/Results_Figures.py

grep -nE \
"Figure 2|noise|isc|GloVe|linguistic|zrep|embedding|transformation|ROI" \
/tmp/Results_Figures.py
```

Die Aufteilung ist:

Notebook/Skript	Aufgabe
`transformer-representations.ipynb`	Erzeugt BERT-Embeddings, Transformationen und GloVe
`transformer-transformation-magnitudes.ipynb`	Erzeugt Transformation Magnitudes
`banded_ridge_regression.py`	Berechnet die Encoding-Modelle
`compute_isc.py`	Berechnet die Noise Ceiling
`Results_Figures.ipynb`	Lädt Encoding- und ISC-Ergebnisse und zeichnet Figure 2

`Layerwise_Plot.ipynb` brauchst du für Figure 2 nicht. Das ist für die layerweisen Analysen von Figure 3.

↳ - All-Layer-Activations erzeugen: (277, 9216). - All-Layer-Z-representation...

aber moment, die habe ich doch schon?

Du hast bereits alle zwölf einzelnen Layer-Dateien:

```
Nur Text

pieman_bert-base-uncased_layer_0_activations.npy
...
pieman_bert-base-uncased_layer_11_activations.npy
```

und:

```
Nur Text

pieman_bert-base-uncased_layer_0_z_representations.npy
...
pieman_bert-base-uncased_layer_11_z_representations.npy
```

Ob du zusätzlich bereits die für Figure 2 benötigten, zusammengefügten All-Layer-Dateien hast, wurde durch dein bisheriges `ls` noch nicht gezeigt. Dort waren nur die einzelnen Layer sichtbar.

Prüfe es mit:

```bash
cd /nethome/phussein/brains/bert-brains

find . -type f \( \
  -iname "*pieman*all*activations*.npy" \
  -o -iname "*pieman*all*z*representations*.npy" \
\) -print
```

Oder etwas allgemeiner:

```bash
find text_processing_and_transformers -maxdepth 1 -type f \
  -iname "*pieman*.npy" |
grep -E "all|full"
```

Wenn keine Ausgabe erscheint, hast du zwar alle benötigten Ausgangsdaten, aber noch nicht die beiden zusammengefügten Dateien.

Wichtig ist der Unterschied:

```text
Vorhanden:
12 einzelne Dateien mit jeweils (277, 768)

Für Figure 2 benötigt:
1 gemeinsame Datei mit (277, 9216)
```

Die gemeinsamen Arrays enthalten keine neuen BERT-Informationen. Sie sind lediglich:

```python
np.concatenate(layer_arrays, axis=1)
```

Du musst also BERT nicht erneut ausführen. Es handelt sich nur um einen kurzen Zusammenfügungsschritt vor dem neuen Encoding-Run.

Falls `find` eine Datei ausgibt, prüfe ihre Form:

```bash
```

```
python - <<'PY'
import glob
import numpy as np

patterns = [
    "text_processing_and_transformers/*pieman*all*activations*.npy",
    "text_processing_and_transformers/*pieman*all*z*representations*.npy",
]

for pattern in patterns:
    for path in glob.glob(pattern):
        x = np.load(path)
        print(path, x.shape, x.dtype)
PY
```

Hat sie `(277, 9216)`, ist dieser Schritt bereits erledigt.

(base) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

git ls-files --error-unmatch \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
error: pathspec 'text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy' did not match any file(s) known to git
Did you forget to 'git add'?
(base) [phussein@submit bert-brains]$ ls -lh --time-style=long-iso \
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_{glove,activations,z_representations}.npy
-rw-r--r-- 1 phussein phussein 832K 2026-08-04 20:47 text_processing_and_transformers/pieman_bert-base-uncased_layer_0_activations.npy
-rw-r--r-- 1 phussein phussein 325K 2026-08-04 20:47 text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
-rw-r--r-- 1 phussein phussein 832K 2026-08-04 20:47 text_processing_and_transformers/pieman_bert-base-uncased_layer_0_z_representations.npy
(base) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

```
find . -type f \( \
  -iname "*pieman*all*activations*.npy" \
  -o -iname "*pieman*all*z*representations*.npy" \
\) -print
(base) [phussein@submit bert-brains]$
```

Genau: Die zwölf einzelnen Layer-Dateien sind vorhanden, aber die zwei zusammengefügten All-Layer-Dateien existieren noch nicht. Deshalb blieb `find` leer.

Du musst BERT nicht erneut laufen lassen. Verbinde nur die vorhandenen Dateien:

Bash

```
cd /nethome/phussein/brains/bert-brains

python - <<'PY'
from pathlib import Path
import numpy as np

feature_dir = Path("text_processing_and_transformers")

activations = []
transformations = []

for layer in range(12):
    a_path = feature_dir / f"pieman_bert-base-uncased_layer_{layer}_activations.npy"
    z_path = feature_dir / f"pieman_bert-base-uncased_layer_{layer}_z_representations.npy"

    a = np.load(a_path)
    z = np.load(z_path)

    print(f"Layer {layer:2d}: activations {a.shape}, z-representations {z.shape}")
```

Erwartete Schlussausgabe:

Nur Text

```
pieman_bert-base-uncased_all_layer_activations.npy (277, 9216) float32
pieman_bert-base-uncased_all_layer_z_represent
```

↳ Deine nächsten konkreten Schritte In dieser Reihenfolge: 1. All-Layer-Acti...

ok. ich mache jetzt mittagspause. lass uns danach hier

weitermachen

Klar, machen wir danach genau hier weiter.

Festgehaltener Stand:

- Alle zwölf einzelnen Activations- und Z-representation-Dateien sind vorhanden.
- Die beiden All-Layer-Dateien existieren noch nicht.
- GloVe ist bereits vorhanden: `(277, 300)`.
- Nächster Schritt: Activations und Z-representations jeweils zu `(277, 9216)` konkatenieren.

Guten Appetit!

letzte frage: ist es ok, das ganze auf den 14 subject dateien zu machen (Stichprobe groß genug?)

Ja – für ein Coding-Projekt sind 14 Subjects ausreichend, um die Pipeline auf Pieman zu reproduzieren und die grundlegenden Muster von Figure 2 zu untersuchen.

Aber es ist keine exakte statistische Reproduktion des Papers, denn dort wurden insgesamt 63 Subjects aus zwei Stories zusammengeführt. Du solltest es daher als „Reproduktion auf dem Pieman-Subset“ bezeichnen.

Mit $N = 14$:

- kannst du Encoding-Modelle und die ISC-Noise-Ceiling sinnvoll berechnen;
- kannst du Median und Bootstrap-Konfidenzintervalle darstellen;
- werden die Konfidenzintervalle deutlich breiter und instabiler sein;
- hast du weniger Power für Unterschiede zwischen Modellen;
- sollten einzelne nicht signifikante Unterschiede nicht als Widerlegung des Papers interpretiert werden.

Wichtig: Die 1000 Parcels erhöhen die Stichprobengröße nicht auf 1000. Die unabhängigen Beobachtungen sind deine 14 Subjects. Modellvergleiche sollten daher subjectweise gepaart sein.

Eine passende Formulierung für den Bericht wäre:

> We reproduced the Figure 2 analysis pipeline using the Pieman story from the Narratives dataset and the 14 available participants. This constitutes a partial reproduction of the original analysis, which aggregated 63 participants across two narrative datasets. The smaller sample is expected to yield wider uncertainty intervals and reduced statistical power.

Mein Urteil: **Für Pipeline-Reproduktion und explorativen Modellvergleich: ja. Für eine belastbare**

Bestätigung aller statistischen Unterschiede aus Figure 2: eher nein.

ok. mittagspause zu ende. was jetzt?

Als Nächstes erzeugen wir aus deinen zwölf vorhandenen Layer-Dateien die zwei All-Layer-Featuredateien für Figure 2. BERT muss dafür nicht erneut laufen.

Führe auf dem Cluster aus:

Bash

```
cd /nethome/phussein/brains/bert-brains

python - <<'PY'
import os
import numpy as np

feature_dir = "text_processing_and_transformers"

activations = []
transformations = []

for layer in range(12):
    act_path = os.path.join(
        feature_dir,
        f"pieman_bert-base-uncased_layer_{layer}_activations.npy"
    )
    z_path = os.path.join(
        feature_dir,
        f"pieman_bert-base-uncased_layer_{layer}_z_representations.npy"
    )
```

Erwartete Schlussausgabe:

Nur Text

```
pieman_bert-base-uncased_all_layer_activations.npy (277, 9216) float32
pieman_bert-base-uncased_all_layer_z_representations.npy (277, 9216) float32
```

Danach kontrollieren:

Bash

```
ls -lh \
text_processing_and_transformers/pieman_bert-base-uncased_all_layer_{activations,z_representations}.npy
```

Poste mir anschließend die Ausgabe. Dann prüfen wir als Nächstes das Original-Notebook für die Transformation Magnitudes, bevor wir weitere Encoding-Runs starten.

Layer 0: activations=(277, 768), z_representations=(277, 768)
Layer 1: activations=(277, 768), z_representations=(277, 768)
Layer 2: activations=(277, 768), z_representations=(277, 768)
Layer 3: activations=(277, 768), z_representations=(277, 768)
Layer 4: activations=(277, 768), z_representations=(277, 768)
Layer 5: activations=(277, 768), z_representations=(277, 768)
Layer 6: activations=(277, 768), z_representations=(277, 768)
Layer 7: activations=(277, 768), z_representations=(277, 768)
Layer 8: activations=(277, 768), z_representations=(277, 768)
Layer 9: activations=(277, 768), z_representations=(277, 768)
Layer 10: activations=(277, 768), z_representations=(277, 768)
Layer 11: activations=(277, 768), z_representations=(277, 768)

Saved:
text_processing_and_transformers/pieman_bert-base-uncased_all_layer_activations.npy (277, 9216) float32
text_processing_and_transformers/pieman_bert-base-uncased_all_layer_z_representations.npy (277, 9216) float32

Sehr gut – die beiden All-Layer-Featuredateien für Figure 2 sind korrekt erzeugt:

```
Nur Text

BERT embeddings:      (277, 9216)
BERT transformations: (277, 9216)
```

Als Nächstes prüfen wir, wie das Original-Notebook die Transformation Magnitudes berechnet. Wir sollten nicht vorschnell nur die bereits TR-gemittelten Z-representations umformen, weil das Paper erst über Tokens mittelt und anschließend die Head-Norm berechnet.

Konvertiere das Notebook in ein lesbares Python-Skript:

```
Bash

cd /nethome/phussein/brains/bert-brains

jupyter nbconvert \
  --to script \
```

```
  text_processing_and_transformers/transformer-transformation-magnitudes.ipynb \
  --output transformer-transformation-magnitudes
```

Danach sollte eine `.py`-Datei entstanden sein. Zeige deren Inhalt:

```bash
sed -n '1,260p' \
text_processing_and_transformers/transformer-transformation-magnitudes.py
```

Falls der Dateiname anders ist, suche ihn:

```bash
find text_processing_and_transformers -maxdepth 1 \
  -iname "*transformation*magnitude*.py" -print
```

Poste mir den ausgegebenen Python-Code. Dann können wir exakt feststellen:

- welche Eingabedateien das Notebook erwartet,
- ob Pieman bereits berücksichtigt wird,
- in welcher Reihenfolge Token-Mittelung und L2-Norm erfolgen,
- welche Ausgabedatei Figure 2 benötigt.

(base) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

jupyter nbconvert \
  --to script \
  text_processing_and_transformers/transformer-transformation-magnitudes.ipynb \
  --output transformer-transformation-magnitudes
-bash: jupyter: command not found
(base) [phussein@submit bert-brains]$ conda activate brains

EnvironmentNameNotFound: Could not find conda environment: brains
You can list all discoverable environments with `conda info --envs`.

```
(base) [phussein@submit bert-brains]$ conda info --envs

# conda environments:
#
# * -> active
# + -> frozen
base                  *  /nethome/phussein/miniconda3
bert-brains              /nethome/phussein/miniconda3/envs/bert-
brains
datalad                  /nethome/phussein/miniconda3/envs/datalad
eap_env                  /nethome/phussein/miniconda3/envs/
eap_env
fmri                     /nethome/phussein/miniconda3/envs/fmri
fmri_encoding            /nethome/phussein/miniconda3/envs/
fmri_encoding
                         /scratch/phussein/envs/noncon_ml
                         /scratch/phussein/envs/pet
                         /scratch/phussein/envs/test
                         /scratch/phussein/envs/test-pet

(base) [phussein@submit bert-brains]$ conda activate bert-
brains
(bert-brains) [phussein@submit bert-brains]$ cd /nethome/
phussein/brains/bert-brains

jupyter nbconvert \
  --to script \
  text_processing_and_transformers/transformer-transformation-
magnitudes.ipynb \
  --output transformer-transformation-magnitudes
[NbConvertApp] Converting notebook
text_processing_and_transformers/transformer-transformation-
magnitudes.ipynb to script
[NbConvertApp] Writing 2300 bytes to
text_processing_and_transformers/transformer-transformation-
magnitudes.py
(bert-brains) [phussein@submit bert-brains]$ sed -n '1,260p' \
text_processing_and_transformers/transformer-transformation-
magnitudes.py
```

```
#!/usr/bin/env python
# coding: utf-8

# In[118]:


import numpy as np

import seaborn as sns
import matplotlib.pylab as plt


# In[128]:


LAYERS = range(0, 12)
MODEL = "bert-base-uncased"
STIMULUS = "black" # slumlordreach, black


# In[129]:


z_reps = {}
for layer in LAYERS:
    z_reps[layer] = np.load("{}_{}_layer_{}
_z_representations.npy".format(STIMULUS, MODEL, layer))

number_of_trs = z_reps[0].shape[0]


# In[130]:


# Indexes to slice out individual heads
indices = [x+64 for x in range(0, 768, 64)]
head_indices = list(zip([0]+ indices, indices))
```

```
# In[131]:


stimulus_responses = []
for i in range(0, number_of_trs):
    z_magnitudes = []
    for layer in LAYERS:
        layer_z_reps = z_reps[layer][i]
#         print(i, layer, layer_z_reps.shape)

        for start, end in head_indices:
            attn_head = layer_z_reps[start:end]
#             print(start, end, attn_head.shape)
            z_magnitudes.append(np.linalg.norm(attn_head))
    stimulus_responses.append(z_magnitudes)


# In[132]:


z_magnitude_array = np.array(stimulus_responses)
z_magnitude_array.shape


# In[133]:


magnitude_output_file = "{}_{}
_zrep_magnitudes.npy".format(STIMULUS, MODEL)
np.save(magnitude_output_file, z_magnitude_array)


# In[134]:


SCP_STRING = "scp {} tsumers@apps.pni.princeton.edu:/
jukebox/griffiths/bert-brains/code/bert-brains/data/{}/{}/
syntactic_analyses"
```

```
scp_formatted = SCP_STRING.format(magnitude_output_file,
STIMULUS, MODEL)
print(scp_formatted)


# In[147]:


tr_to_compare = 533

attn_scores = np.load("{}_{}_syntactic_complexity_L-inf_T-128_D-
bck.npy".format(STIMULUS, MODEL), allow_pickle=True)
attn_square_matrix =
np.array(attn_scores[tr_to_compare]).reshape(12,12)
sns.heatmap(attn_square_matrix)
plt.suptitle("Attention Scores")

plt.figure()
z_magnitudes = np.load(magnitude_output_file,
allow_pickle=True)
z_square_matrix =
np.array(z_magnitudes[tr_to_compare]).reshape(12,12)
sns.heatmap(z_square_matrix)
plt.suptitle("Z Magnitudes")


# In[148]:


for i in range(50, 500):
    attn_scores_tr = attn_scores[i]
    if attn_scores_tr is None:
        continue

    attn_magnitude = np.linalg.norm(attn_scores_tr)
    zs_magnitude = np.linalg.norm(z_magnitudes[i])
    print("{:.2f} | {:.2f}".format(attn_magnitude, zs_magnitude))
```

```
# In[ ]:



(bert-brains) [phussein@
```

Das Notebook bestätigt die Berechnung eindeutig:

1. Es lädt die zwölf Dateien `layer_X_z_representations.npy`.
2. Jeder 768-dimensionale Layervektor wird in zwölf Abschnitte à 64 Dimensionen geteilt.
3. Für jeden Head wird die L2-Norm berechnet.
4. Alle zwölf Layer × zwölf Heads werden zu 144 Features verbunden.

Damit können wir die Pieman-Datei exakt nach der Notebook-Logik erzeugen. Das vollständige Notebook solltest du nicht einfach ausführen, weil seine späteren Visualisierungszellen für `black` geschrieben sind und unter anderem auf `TR 533` zugreifen – Pieman hat nur 277 TRs.

Führe stattdessen aus:

```bash
cd /nethome/phussein/brains/bert-brains/text_processing_and_transformers

python - <<'PY'
import numpy as np

LAYERS = range(12)
MODEL = "bert-base-uncased"
STIMULUS = "pieman"

# Genau wie im Original-Notebook
z_reps = {}

for layer in LAYERS:
    path = f"{STIMULUS}_{MODEL}_layer_{layer}_z_representations.npy"
    z_reps[layer] = np.load(path)

    if z_reps[layer].shape != (277, 768):
        raise ValueError(
            f"Unexpected shape for layer {layer}: {z_reps[layer].shape}"
        )
```

Erwartet:

```
Saved: pieman_bert-base-uncased_zrep_magnitudes.npy
Shape: (277, 144)
Dtype: float32
NaNs: 0
Infs: 0
```

Danach hast du vier der fünf Figure-2-Featurebedingungen vorbereitet:

Featurebedingung	Status
GloVe `(277, 300)`	fertig
BERT embeddings `(277, 9216)`	fertig
BERT transformations `(277, 9216)`	fertig
Transformation magnitudes `(277, 144)`	nach diesem Befehl fertig
Linguistic features	noch zu prüfen/erzeugen

Poste anschließend die Ausgabe. Dann suchen wir gezielt nach den linguistischen Pieman-Features.

mach mir das als python skript zum runnen

Dauer: 49 Sek. >

Hier ist das fertige Skript:

Lade es auf den Cluster in das Repo-Hauptverzeichnis und führe es dort aus:

```bash
cd /nethome/phussein/brains/bert-brains
python create_pieman_transformation_magnitudes.py
```

Das Skript findet automatisch `text_processing_and_transformers/`, lädt alle zwölf Z-representation-Dateien und erzeugt:

```
```

```
text_processing_and_transformers/pieman_bert-base-uncased_zrep_magnitudes.npy
```

Erwartete Form:

```
Nur Text

(277, 144)
```

Es prüft außerdem:

- alle zwölf Eingabedateien,
- Shape `(277, 768)` pro Layer,
- NaNs und unendliche Werte,
- das fertige Ausgabeformat.

create_pieman_transformation_magnitudes.py
Code

(bert-brains) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains
python create_pieman_transformation_magnitudes.py
Layer 0: (277, 768)
Layer 1: (277, 768)
Layer 2: (277, 768)
Layer 3: (277, 768)
Layer 4: (277, 768)
Layer 5: (277, 768)
Layer 6: (277, 768)
Layer 7: (277, 768)
Layer 8: (277, 768)
Layer 9: (277, 768)
Layer 10: (277, 768)
Layer 11: (277, 768)

Saved: /nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_bert-base-uncased_zrep_magnitudes.npy
Shape: (277, 144)
Dtype: float32
NaNs: 0
Infs: 0

Min: 0.11820387840270996
Max: 7.461196422576904

Perfekt. Die Transformation Magnitudes sind korrekt erzeugt:

```
Nur Text

Shape: (277, 144)
NaNs/Infs: 0
```

Damit sind jetzt vier der fünf Figure-2-Featurebedingungen fertig:

- GloVe
- BERT embeddings, alle Layer
- BERT transformations, alle Layer
- transformation magnitudes

Jetzt prüfen wir, ob die linguistischen Pieman-Features bereits existieren:

```
Bash

cd /nethome/phussein/brains/bert-brains

find . -type f \( \
  -iname "*pieman*pos*.npy" \
  -o -iname "*pieman*dep*.npy" \
  -o -iname "*pieman*linguistic*.npy" \
  -o -iname "*pieman*syntax*.npy" \
\) -print
```

Da das Originalskript eventuell Dateien nur nach einzelnen Dependency-Namen benennt, zusätzlich:

```
Bash

find data text_processing_and_transformers -type f -name "*.npy" |
grep -Ei \
"pieman|prep|pobj|det|nsubj|amod|dobj|advmod|aux|poss|ccomp|mark|prt|pos"
```

Zeige außerdem den Code des Extraktionsskripts:

```
Bash

sed -n '1,260p' \
```

```
text_processing_and_transformers/extract_linguistic_features.py
```

Poste mir diese Ausgaben. Danach können wir feststellen, ob die linguistischen Features nur gefunden, für Pieman neu erzeugt oder zu einer gemeinsamen Figure-2-Matrix kombiniert werden müssen.

(bert-brains) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

```
find . -type f \( \
  -iname "*pieman*pos*.npy" \
  -o -iname "*pieman*dep*.npy" \
  -o -iname "*pieman*linguistic*.npy" \
  -o -iname "*pieman*syntax*.npy" \
\) -print
```

(bert-brains) [phussein@submit bert-brains]$

Damit wurden keine offensichtlich benannten linguistischen Pieman-Featuredateien gefunden. Bevor wir sie als fehlend einstufen, prüfen wir die tatsächlichen Dateinamen und das Extraktionsskript.

Führe zuerst aus:

Bash

```
cd /nethome/phussein/brains/bert-brains

find data text_processing_and_transformers \
  -type f -name "*.npy" |
grep -Ei \
"prep|pobj|det|nsubj|amod|dobj|advmod|aux|poss|ccomp|mark|prt|pos|depend"
```

Danach bitte das Skript anzeigen:

Bash

```
sed -n '1,300p' \
text_processing_and_transformers/extract_linguistic_features.py
```

Poste beide Ausgaben. Daraus sehen wir:

- welche Transkriptdatei das Skript erwartet,

- ob Pieman bereits unterstützt wird,
- welche POS- und Dependency-Dateien es erzeugt,
- ob Figure 2 eine gemeinsame Matrix oder mehrere einzelne Featuredateien verwendet.

(bert-brains) [phussein@submit bert-brains]$ find data text_processing_and_transformers -type f -name "*.npy" | grep -Ei \
"pieman|prep|pobj|det|nsubj|amod|dobj|advmod|aux|poss|ccomp|mark|prt|pos"
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_1_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_1_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_2_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_2_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_3_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_3_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_4_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_4_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_5_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_5_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_6_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_6_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_7_activations.npy

text_processing_and_transformers/pieman_bert-base-uncased_layer_7_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_8_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_8_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_9_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_9_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_10_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_10_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_11_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_11_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_layer_0_glove.npy
text_processing_and_transformers/pieman_bert-base-uncased_headwise-distance_T-128_D-bck.npy
text_processing_and_transformers/raw_pieman_bert-base-uncased_headwise-distance_T-128_D-bck.npy
text_processing_and_transformers/pieman_bert-base-uncased_all_layer_activations.npy
text_processing_and_transformers/pieman_bert-base-uncased_all_layer_z_representations.npy
text_processing_and_transformers/pieman_bert-base-uncased_zrep_magnitudes.npy
(bert-brains) [phussein@submit bert-brains]$

Damit ist bestätigt: In `data/` und `text_processing_and_transformers/` existieren aktuell keine linguistischen Pieman-Features.

Vorhanden sind jetzt:

```
Nur Text

GloVe                          (277, 300)
```

```
BERT embeddings, alle Layer    (277, 9216)
BERT transformations           (277, 9216)
Transformation magnitudes      (277, 144)
```

Es fehlt nur noch die fünfte Figure-2-Featureklasse:

```
Nur Text

POS + Dependency Features
```

Jetzt müssen wir sehen, welche Eingabedaten `extract_linguistic_features.py` benötigt. Bitte führe aus:

```
Bash

cd /nethome/phussein/brains/bert-brains

sed -n '1,320p' \
text_processing_and_transformers/extract_linguistic_features.py
```

Falls das Skript länger ist:

```
Bash

sed -n '321,640p' \
text_processing_and_transformers/extract_linguistic_features.py
```

Poste den Code. Danach passe ich ihn gezielt für `pieman` an.

kann ich dir einfach die datei hochladen?

Ja, lade `extract_linguistic_features.py` einfach hier hoch. Dann kann ich direkt prüfen:

- welche Eingabedateien erwartet werden,
- ob Pieman bereits unterstützt wird,
- welche POS- und Dependency-Features erzeugt werden,
- welche Anpassungen für Figure 2 nötig sind.

Falls das Skript weitere lokale Dateien importiert, lade die danach bei Bedarf ebenfalls hoch.

extract_linguistic_f...
Code

brauchst du weitere skripte oder infos?

Dauer: 14 Sek. >

Ja. Für die linguistischen Features brauche ich noch drei Dateien:

1. `decode_linguistic_features_utils.py`
   Das Skript importiert daraus `LoadLingFeatures`, daher müssen wir prüfen, welche Pfade und Formate diese Funktion erwartet.
2. `data/pieman/align.csv`
3. `data/pieman/tr_tokens.csv`

Prüfe zunächst, ob die beiden CSV-Dateien existieren:

```bash
cd /nethome/phussein/brains/bert-brains

ls -lh \
data/pieman/align.csv \
data/pieman/tr_tokens.csv
```

Falls der Pfad nicht stimmt:

```bash
find . -type f \( \
  -name "align.csv" \
  -o -name "tr_tokens.csv" \
\) -print
```

Lade anschließend möglichst diese drei Dateien hoch:

```
text_processing_and_transformers/decode_linguistic_features_utils.py
data/pieman/align.csv
data/pieman/tr_tokens.csv
```

Warum das nötig ist: `extract_linguistic_features.py` unterstützt Pieman aktuell noch nicht:

```python
```

```
choices=['black', 'slumlordreach']
```

Außerdem erzeugt es am Ende:

```
Nur Text

pieman_ling_features.npz
```

mit 39 Features:

- 25 Dependency-Labels
- 14 POS-Labels

Wir müssen noch prüfen, ob `LoadLingFeatures` mit Pieman funktioniert und ob die Tokenanzahl zwischen `align.csv` und `tr_tokens.csv` korrekt übereinstimmt. Danach kann ich dir ein angepasstes, direkt ausführbares Pieman-Skript erstellen.

Für den späteren Figure-2-Schritt wären zusätzlich hilfreich:

- dein aktuelles `banded_ridge_regression.py`
- dein aktuelles `run_encoding_pieman.py`
- `Results_Figures.ipynb`

Aber die brauchen wir erst nach der Erzeugung der linguistischen Features.

decode_linguistic_f... Code

decode_linguistic_f... Code

Align

Tr Tokens

Dauer: 28 Sek.

Für die linguistischen Features brauche ich keine weiteren Dateien. Die beiden CSV-Dateien passen zusammen:

- `align.csv`: 957 Wörter
- `tr_tokens.csv`: 277 TRs
- Summe der Wörter über alle TRs: 957
- 254 TRs mit Text, 23 leere TRs

Damit ist die zeitliche Zuordnung vollständig konsistent.

Allerdings ist das originale `extract_linguistic_features.py` in seinem aktuellen Zustand nicht ausführbar. Am Ende verwendet es vier Variablen, die vorher nie definiert werden:

```python
dep_features
pos_features
dep_list
pos_list
```

Vermutlich fehlten beim Export aus einem Notebook einige Zellen. `LoadLingFeatures` wird zwar importiert, aber nie aufgerufen. Ein unveränderter Lauf würde daher mit einem `NameError` abbrechen.

Außerdem akzeptiert das Skript bislang nur:

```python
choices=["black", "slumlordreach"]
```

und nicht `pieman`.

`decode_linguistic_features.py` brauchen wir für Figure 2 nicht. Dieses Skript führt die umgekehrte Analyse durch – es sagt linguistische Labels aus einzelnen Attention Heads vorher und gehört zu späteren Head-Spezialisierungsanalysen.

Für den aktuellen Schritt kann ich jetzt ein repariertes Skript erstellen, das:

- `pieman` unterstützt,
- `align.csv` und `tr_tokens.csv` lädt,
- spaCy auf das gesamte Transkript anwendet,
- die Tokenausrichtung überprüft,
- POS- und Dependency-Häufigkeiten pro TR berechnet,
- `pieman_ling_features.npz` speichert,
- zusätzlich eine direkt für das Encoding geeignete `.npy`-Matrix erzeugt.

Für den anschließenden Encoding- und Plot-Schritt brauche ich später noch deine aktuellen Versionen von:

```
Nur Text

analysis/banded_ridge_regression.py
analysis/run_encoding_pieman.py
visualizations/Results_Figures.ipynb
```

Aber erst nachdem wir die linguistischen Features erzeugt haben.

ja bitte

Du hast nach 0 Sek. gestoppt

Results_Figures.ipy... Code | banded_ridge_regr... Code | run_encoding_pie... Code

Dauer: 1 Min. 30 Sek. >

Hier ist das fertige Skript:

Lade es in das Hauptverzeichnis deines Repos und führe es aus:

```
Bash

cd /nethome/phussein/brains/bert-brains
```

```
conda activate bert-brains

python create_pieman_linguistic_features.py
```

Es erzeugt insbesondere:

```
Nur Text

text_processing_and_transformers/pieman_ling_features.npy
text_processing_and_transformers/pieman_ling_features.npz
```

Die erwartete Encoding-Matrix ist:

```
Nur Text

(277, 39)
```

Zusätzlich werden zur Kontrolle erzeugt:

```
Nur Text

data/pieman/ling_features.csv
data/pieman/tr_tokens_new.csv
```

Falls das spaCy-Modell fehlt:

```
Bash

python -m spacy download en_core_web_lg
```

Poste mir anschließend bitte die komplette Ausgabe. Dann prüfen wir die Matrix und passen `run_encoding_pieman.py` für die fünf Figure-2-Bedingungen an.

create_pieman_linguistic_features.py
Code

soll ich es vllt lieber als condor job einreichen?

Nein, dafür brauchst du keinen Condor-Job. Das Skript verarbeitet nur 957 Wörter mit spaCy und erzeugt eine kleine Matrix `(277, 39)`. Das sollte auf dem Submit-Node höchstens wenige Sekunden bis etwa eine Minute dauern und benötigt keine GPU.

Führe es direkt aus:

```bash
cd /nethome/phussein/brains/bert-brains
conda activate bert-brains
python create_pieman_linguistic_features.py
```

Nur falls eure Clusterregeln auch kurze CPU-Berechnungen auf dem Submit-Node ausdrücklich verbieten, solltest du Condor verwenden. Die späteren 70 Encoding-Runs gehören dagegen definitiv in Condor.

conda activate bert-brains

python create_pieman_linguistic_features.py
Traceback (most recent call last):
  File "/nethome/phussein/brains/bert-brains/create_pieman_linguistic_features.py", line 187, in <module>
    main()
  File "/nethome/phussein/brains/bert-brains/create_pieman_linguistic_features.py", line 85, in main
    raise FileNotFoundError(f"Missing input file: {path}")
FileNotFoundError: Missing input file: /nethome/phussein/brains/bert-brains/data/pieman/align.csv
(bert-brains) [phussein@submit bert-brains]$

Das Skript funktioniert grundsätzlich, aber die Pieman-CSV-Dateien liegen auf deinem Cluster nicht unter dem erwarteten Pfad:

```
data/pieman/align.csv
data/pieman/tr_tokens.csv
```

Suche zunächst ihre tatsächlichen Pfade:

```bash
cd /nethome/phussein/brains/bert-brains

find . -type f \( \
```

```
  -name "align.csv" \
  -o -name "tr_tokens.csv" \
\) -print
```

Wahrscheinlich findest du mehrere Versionen für andere Geschichten, aber noch keine unter `data/pieman/`. Falls die Pieman-Dateien beispielsweise woanders liegen, kopiere sie in den erwarteten Ordner:

```bash
mkdir -p data/pieman

cp /TATSÄCHLICHER/PFAD/align.csv data/pieman/align.csv
cp /TATSÄCHLICHER/PFAD/tr_tokens.csv data/pieman/tr_tokens.csv
```

Danach kontrollieren:

```bash
ls -lh data/pieman/{align.csv,tr_tokens.csv}
```

und erneut ausführen:

```bash
python create_pieman_linguistic_features.py
```

Poste am besten zuerst die Ausgabe des `find`-Befehls, damit wir sicher die richtigen Pieman-Dateien nehmen und nicht versehentlich `black` oder `slumlordreach`.

ileNotFoundError: Missing input file: /nethome/phussein/brains/bert-brains/data/pieman/align.csv
(bert-brains) [phussein@submit bert-brains]$ cd /nethome/phussein/brains/bert-brains

find . -type f \(\
  -name "align.csv" \
  -o -name "tr_tokens.csv" \
\) -print
./data/21st_year/tr_tokens.csv
./data/stimuli/black/align.csv

./data/stimuli/black/tr_tokens.csv
./data/stimuli/bronx/align.csv
./data/stimuli/forgot/align.csv
./data/stimuli/lucy/align.csv
./data/stimuli/merlin/align.csv
./data/stimuli/milkywayoriginal/align.csv
./data/stimuli/milkywaysynonyms/align.csv
./data/stimuli/milkywayvodka/align.csv
./data/stimuli/notthefallintact/align.csv
./data/stimuli/pieman/align.csv
./data/stimuli/pieman/tr_tokens.csv
./data/stimuli/piemanpni/align.csv
./data/stimuli/prettymouth/align.csv
./data/stimuli/shapesphysical/align.csv
./data/stimuli/shapessocial/align.csv
./data/stimuli/sherlock/align.csv
./data/stimuli/slumlordreach/align.csv
./data/stimuli/slumlordreach/tr_tokens.csv

Die richtigen Dateien sind vorhanden; das Skript erwartet lediglich den falschen Zwischenpfad.

Vorhanden:

```
Nur Text

data/stimuli/pieman/align.csv
data/stimuli/pieman/tr_tokens.csv
```

Erwartet:

```
Nur Text

data/pieman/...
```

Am einfachsten erstellst du einen symbolischen Link:

```
Bash

cd /nethome/phussein/brains/bert-brains

ln -s stimuli/pieman data/pieman
```

Kontrolliere ihn:

```bash
ls -lh data/pieman/{align.csv,tr_tokens.csv}
```

Danach erneut:

```bash
python create_pieman_linguistic_features.py
```

Der Link dupliziert keine Dateien. Er sorgt nur dafür, dass der vom Original-Repo verwendete Pfad `data/pieman/` auf deine tatsächlich vorhandenen Dateien unter `data/stimuli/pieman/` verweist.

bin vom submit knoten geflogen. ich sollte einen job machen.

Dauer: 19 Sek. >

Ja, dann machen wir es als kleinen CPU-Condor-Job. Meine Empfehlung, es direkt auf dem Submit-Knoten auszuführen, war für euren Cluster nicht passend – sorry.

Zuerst korrigierst du im Python-Skript den Datenpfad:

```bash
cd /nethome/phussein/brains/bert-brains

sed -i \
  's|repo_root / "data" / "pieman"|repo_root / "data" / "stimuli" / "pieman"|' \
  create_pieman_linguistic_features.py
```

Prüfen:

```bash
grep -n "data_dir =" create_pieman_linguistic_features.py
```

Die Ausgabe sollte sein:

```python
data_dir = repo_root / "data" / "stimuli" / "pieman"
```

Erstelle dann `/nethome/phussein/brains/pieman_linguistic_features.sh`:

```bash
nano /nethome/phussein/brains/pieman_linguistic_features.sh
```

Inhalt:

```bash
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains

echo "Starting Pieman linguistic feature extraction"
python create_pieman_linguistic_features.py
echo "Finished successfully"
```

Ausführbar machen:

```bash
chmod +x /nethome/phussein/brains/pieman_linguistic_features.sh
```

Dann `/nethome/phussein/brains/pieman_linguistic_features.sub` erstellen:

```bash
nano /nethome/phussein/brains/pieman_linguistic_features.sub
```

Inhalt:

```
initialdir = /nethome/phussein/brains/bert-brains
executable = /nethome/phussein/brains/pieman_linguistic_features.sh
```

```
output = /scratch/phussein/logs/brains/pieman_ling.$(ClusterId).out
error  = /scratch/phussein/logs/brains/pieman_ling.$(ClusterId).err
log    = /scratch/phussein/logs/brains/pieman_ling.$(ClusterId).log

request_CPUs   = 1
request_memory = 8G

getenv = True

queue 1
```

Log-Verzeichnis sicherstellen und Job absenden:

```bash
mkdir -p /scratch/phussein/logs/brains

cd /nethome/phussein/brains
condor_submit pieman_linguistic_features.sub
```

Status prüfen:

```bash
condor_q
```

Nach Abschluss:

```bash
cat /scratch/phussein/logs/brains/pieman_ling.*.out
cat /scratch/phussein/logs/brains/pieman_ling.*.err
```

Bei Erfolg sollte die Ausgabe unter anderem zeigen:

```
Shape: (277, 39)
NaNs: 0
Finished successfully
```

Die relevante Datei liegt danach hier:

```
/nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_ling_features.npy
```

ich mache es in brains.sh

Ja, das geht. Ersetze den bisherigen Inhalt von `brains.sh` für diesen Lauf durch:

```bash
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains

echo "Starting Pieman linguistic feature extraction"
python create_pieman_linguistic_features.py
echo "Finished successfully"
```

Vorher noch den Pfad im Python-Skript korrigieren:

```bash
cd /nethome/phussein/brains/bert-brains

sed -i \
  's|repo_root / "data" / "pieman"|repo_root / "data" / "stimuli" / "pieman"|' \
  create_pieman_linguistic_features.py
```

Für `brains.sub` brauchst du hierfür keine GPU. Ändere den Ressourcenblock zu:

```
request_CPUs   = 1
request_memory = 8G
```

Entferne beziehungsweise kommentiere diese Zeilen:

```
request_GPUs = 1
```

```
requirements = (GPUs_Capability >=8.0)&&(GPUs_GlobalMemoryMb > 60000)
```

Dann:

```bash
chmod +x /nethome/phussein/brains/brains.sh

cd /nethome/phussein/brains
condor_submit brains.sub
```

Nach Abschluss findest du die Ausgabe mit:

```bash
cat /scratch/phussein/logs/brains/brains.sh.*.out
cat /scratch/phussein/logs/brains/brains.sh.*.err
```

Erwartetes Ergebnis:

```
text_processing_and_transformers/pieman_ling_features.npy
Shape: (277, 39)
```

```
#!/bin/bash

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

conda activate bert-brains

cd /nethome/phussein/brains/bert-brains

echo "Starting Pieman linguistic feature extraction"

python create_pieman_linguistic_features.py

echo "Finished successfully"
```

Ja, `brains.sh` ist fast richtig. Ergänze nur `set -e`, damit der Job bei einem Python-Fehler

abbricht und nicht fälschlich „Finished successfully“ ausgibt:

```bash
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate bert-brains

cd /nethome/phussein/brains/bert-brains

echo "Starting Pieman linguistic feature extraction"

python create_pieman_linguistic_features.py

echo "Finished successfully"
```

Wichtig: Kontrolliere vorher, ob der korrigierte Pfad im Python-Skript steht:

```bash
grep -n "data_dir =" \
/nethome/phussein/brains/bert-brains/create_pieman_linguistic_features.py
```

Erwartet:

```python
data_dir = repo_root / "data" / "stimuli" / "pieman"
```

Anschließend `brains.sub` ohne GPU absenden:

```bash
chmod +x /nethome/phussein/brains/brains.sh
cd /nethome/phussein/brains
condor_submit brains.sub
```

ok, läuft gerade. wir extracten gerade die linguistic features, ja?

Ja, genau. Der Job extrahiert gerade die linguistischen Features für Pieman.

Er macht Folgendes:

- parst das Pieman-Transkript mit spaCy,
- bestimmt für jedes Wort POS-Tag und Dependency-Relation,
- ordnet die Wörter den 277 TRs zu,
- zählt pro TR die 25 Dependency- und 14 POS-Kategorien,
- erzeugt dadurch eine Matrix mit:

```
Nur Text

277 TRs × 39 linguistische Features
```

Die wichtigste Ausgabedatei für das Encoding-Modell ist:

```
Nur Text

text_processing_and_transformers/pieman_ling_features.npy
```

Zusätzlich erzeugt er:

```
Nur Text

text_processing_and_transformers/pieman_ling_features.npz
data/stimuli/pieman/ling_features.csv
data/stimuli/pieman/tr_tokens_new.csv
```

Wenn der Job fertig ist, prüfen wir die Logausgabe und danach alle fünf Figure-2-Featuredateien gemeinsam.

(base) [phussein@login brains]$ cat brains.sh.112700.2026_09_03_1788439720.out
Starting Pieman linguistic feature extraction
Repository: /nethome/phussein/brains/bert-brains
Words in align.csv: 957
TRs: 277 (254 with text)
Aligned spaCy tokens: 978
Saved encoding matrix: /nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_ling_features.npy
Saved labelled archive: /nethome/phussein/brains/bert-brains/text_processing_and_transformers/pieman_ling_features.npz

Shape: (277, 39)
Dtype: float32
NaNs: 0
Finished successfully
(base) [phussein@login brains]$ cat
brains.sh.112700.2026_09_03_1788439720.err
(base) [phussein@login brains]$

Der Job ist erfolgreich durchgelaufen. Die linguistischen Features sind fertig:

```
Nur Text

pieman_ling_features.npy
Shape: (277, 39)
NaNs: 0
```

Die `39` Features entsprechen:

- 25 Dependency-Relationen
- 14 POS-Kategorien

Dass aus 957 Wörtern 978 spaCy-Tokens werden, ist plausibel: spaCy zerlegt manche Einträge zusätzlich, etwa Kontraktionen oder Satzzeichen. Entscheidend ist, dass Gesamttranskript und TR-Texte mit demselben Tokenizer jeweils 978 Tokens ergeben; deshalb hat die interne Ausrichtungsprüfung bestanden.

Damit sind jetzt alle fünf Figure-2-Featureklassen vorhanden:

Bedingung	Erwartete Form
Linguistic features	`(277, 39)`
GloVe	`(277, 300)`
Transformation magnitudes	`(277, 144)`
BERT embeddings, alle Layer	`(277, 9216)`
BERT transformations, alle Layer	`(277, 9216)`

Prüfe sie gesammelt:

```bash
cd /nethome/phussein/brains/bert-brains

python - <<'PY'
from pathlib import Path
import numpy as np

feature_dir = Path("text_processing_and_transformers")

files = [
    "pieman_ling_features.npy",
    "pieman_bert-base-uncased_layer_0_glove.npy",
    "pieman_bert-base-uncased_zrep_magnitudes.npy",
    "pieman_bert-base-uncased_all_layer_activations.npy",
    "pieman_bert-base-uncased_all_layer_z_representations.npy",
]

for name in files:
    x = np.load(feature_dir / name)
    print(f"{name:65s} {str(x.shape):14s} finite={np.isfinite(x).all()}")
PY
```

Danach passen wir `run_encoding_pieman.py` so an, dass genau diese fünf Bedingungen für alle 14 Subjects gerechnet werden.

```
pieman_ling_features.npy                                          (277, 39)
finite=True
pieman_bert-base-uncased_layer_0_glove.npy
(277, 300)     finite=True
pieman_bert-base-uncased_zrep_magnitudes.npy
(277, 144)     finite=True
pieman_bert-base-uncased_all_layer_activations.npy
(277, 9216)    finite=True
pieman_bert-base-uncased_all_layer_z_representations.npy
(277, 9216)    finite=True
(base) [phussein@login bert-brains]$
```

Dauer: 28 Sek. >

Perfekt: Alle fünf Featuredateien sind vollständig vorbereitet. Der nächste Schritt sind die Figure-2-Encoding-Modelle.

Wichtig: Verwende dafür ein separates Skript, damit du nicht versehentlich die bereits berechneten 24 layerweisen Figure-3-Runs wiederholst.

```bash
cd /nethome/phussein/brains/bert-brains/analysis

cp run_encoding_pieman.py run_encoding_pieman_figure2.py
nano run_encoding_pieman_figure2.py
```

Ersetze darin den gesamten Abschnitt, der `representations` erstellt, durch:

```python
# ------------------------------------------------
# Figure 2 representations
# ------------------------------------------------

representations = [
    (
        "linguistic",
        "pieman_ling_features.npy",
    ),
    (
        "glove",
        "pieman_bert-base-uncased_layer_0_glove.npy",
    ),
    (
        "zrep_magnitudes",
        "pieman_bert-base-uncased_zrep_magnitudes.npy",
    ),
    (
        "all_layer_activations",
        "pieman_bert-base-uncased_all_layer_activations.npy",
```

Der restliche Aufruf bleibt gleich. Pro Subject werden dann genau fünf Runs erzeugt:

```
results/pieman_encoding_linguistic/
results/pieman_encoding_glove/
results/pieman_encoding_zrep_magnitudes/
results/pieman_encoding_all_layer_activations/
results/pieman_encoding_all_layer_z_representations/
```

In `fmri_encoding.sh` muss anschließend diese Zeile:

```bash
```

```
python run_encoding_pieman.py sub-001
```

entsprechend zu:

```bash
python run_encoding_pieman_figure2.py sub-001
```

geändert werden – ebenso für die übrigen 13 Subjects.

Das ergibt insgesamt:

```
14 Subjects × 5 Featureklassen = 70 Encoding-Runs
```

Diese Berechnung gehört auf jeden Fall in Condor. Besonders die beiden All-Layer-Bedingungen sind wegen der vier HRF-Verzögerungen sehr groß.

werden jetzt die alten ergebnisse überschrieben?

Nein – wenn du das separate `run_encoding_pieman_figure2.py` mit den neuen Namen verwendest, werden die alten layerweisen Ergebnisse nicht überschrieben.

Die vorhandenen Figure-3-Ergebnisse liegen beispielsweise in:

```
results/pieman_encoding_layer_0_activations/
results/pieman_encoding_layer_0_z_representations/
...
results/pieman_encoding_layer_11_z_representations/
```

Die neuen Figure-2-Ergebnisse landen dagegen in:

```
results/pieman_encoding_linguistic/
results/pieman_encoding_glove/
results/pieman_encoding_zrep_magnitudes/
results/pieman_encoding_all_layer_activations/
results/pieman_encoding_all_layer_z_representations/
```

Die Ordnernamen sind verschieden, daher bleiben die alten Ergebnisse erhalten.

Beachte aber: Wenn du denselben Figure-2-Job später noch einmal startest, werden die Figure-2-Dateien desselben Subjects überschrieben, weil `np.save(...)` dieselben Dateinamen verwendet. Das betrifft dann beispielsweise:

```
Nur Text

results/pieman_encoding_glove/sub-001_parcelwise_results_banded_ridge.npy
```

Vor dem Start kannst du sicherheitshalber prüfen:

```
Bash

cd /nethome/phussein/brains/bert-brains

find results -maxdepth 1 -type d \
  -name "pieman_encoding_*" |
sort
```

Solange du die neuen, eindeutig benannten Ordner verwendest, bleiben deine layerweisen Ergebnisse unangetastet.

top, heißt ich lasse jetzt den job laufen?

Ja – sofern du `run_encoding_pieman_figure2.py` erstellt und in `fmri_encoding.sh` alle Aufrufe darauf umgestellt hast, kannst du den Condor-Job starten.

Vorher kurz kontrollieren:

```
Bash

cd /nethome/phussein/brains/bert-brains

grep -n "representations\|linguistic\|glove\|zrep_magnitudes\|all_layer" \
analysis/run_encoding_pieman_figure2.py

grep -n "run_encoding" \
/nethome/phussein/brains/fmri_encoding.sh
```

In `fmri_encoding.sh` sollten alle 14 Subjects mit diesem Skript aufgerufen werden:

```bash
python run_encoding_pieman_figure2.py sub-001
...
python run_encoding_pieman_figure2.py sub-016
```

Dann absenden:

```bash
cd /nethome/phussein/brains
condor_submit fmri_encoding.sub
```

Status:

```bash
condor_q
```

Ein wichtiger Punkt: Dein Shell-Skript verarbeitet die 14 Subjects nacheinander; innerhalb jedes Subjects laufen fünf Modelle. Das ist ein einzelner, potenziell sehr langer Job. Er benötigt keine GPU, aber wegen der All-Layer-Matrizen würde ich mindestens diese Ressourcen verwenden:

```
request_CPUs   = 4
request_memory = 32G
```

Nach dem Start kannst du das Live-Log prüfen:

```bash
tail -f /scratch/phussein/logs/brains/fmri_encoding.<JOB-ID>.out
```

Der Job berechnet dann 70 neue Figure-2-Encoding-Ergebnisse, ohne die layerweisen Ergebnisse zu überschreiben.

Do., 3. Sept. um 16:15

wie/wo kann ich die zwischenergebnisse sehen?

Du kannst die Zwischenergebnisse an zwei Stellen sehen: im Condor-Log und direkt in den Ergebnisordnern.

Den aktuell laufenden Job findest du mit:

```bash
condor_q
```

Die laufende Textausgabe ansehen:

```bash
tail -f /scratch/phussein/logs/brains/fmri_encoding.<CLUSTER-ID>.out
```

Ersetze `<CLUSTER-ID>` durch die ID aus `condor_q`, beispielsweise:

```bash
tail -f /scratch/phussein/logs/brains/fmri_encoding.112701.out
```

Beenden der Live-Anzeige mit `Ctrl+C` stoppt nur `tail`, nicht den Job.

Fehlermeldungen kontrollieren:

```bash
tail -f /scratch/phussein/logs/brains/fmri_encoding.<CLUSTER-ID>.err
```

Bereits fertige Resultate werden unmittelbar hier gespeichert:

```
/nethome/phussein/brains/bert-brains/results/
```

Anzahl fertiger Scoredateien pro Bedingung:

```bash
cd /nethome/phussein/brains/bert-brains

for condition in \
  linguistic \
```

```
  glove \
  zrep_magnitudes \
  all_layer_activations \
  all_layer_z_representations
do
    dir="results/pieman_encoding_${condition}"
    count=$(find "$dir" -maxdepth 1 \
        -name "sub-*_parcelwise_results_banded_ridge.npy" \
        2>/dev/null | wc -l)
    echo "$condition: $count / 14 Subjects"
done
```

Die vorhandenen Dateien anzeigen:

```bash
find results/pieman_encoding_* \
  -name "sub-*_parcelwise_results_banded_ridge.npy" |
sort
```

Da dein Job pro Subject die fünf Bedingungen nacheinander verarbeitet, solltest du die Ergebnisse ungefähr in dieser Reihenfolge sehen:

```
sub-001 linguistic
sub-001 glove
sub-001 zrep_magnitudes
sub-001 all_layer_activations
sub-001 all_layer_z_representations
sub-002 linguistic
...
```

Beachte: Eine Datei wird erst geschrieben, wenn der jeweilige Encoding-Run vollständig beendet ist. Während der Berechnung eines einzelnen Modells entsteht daher noch keine teilweise Scoredatei.

done
linguistic: 2 / 14 Subjects
glove: 2 / 14 Subjects
zrep_magnitudes: 2 / 14 Subjects
all_layer_activations: 2 / 14 Subjects
all_layer_z_representations: 1 / 14 Subjects

Der Job läuft korrekt. Der aktuelle Stand bedeutet:

- `sub-001`: alle fünf Bedingungen fertig
- `sub-002`: die ersten vier Bedingungen fertig
- `sub-002`: `all_layer_z_representations` läuft wahrscheinlich gerade

Damit sind momentan **9 von 70 Encoding-Runs** abgeschlossen.

Du kannst die laufende Bedingung im Log prüfen:

```bash
tail -n 30 /scratch/phussein/logs/brains/fmri_encoding.<CLUSTER-ID>.out
```

Den Fortschritt weiterhin mit deinem Zählbefehl beobachten. Solange:

```bash
condor_q
```

den Job als `R` anzeigt und die `.err`-Datei leer bleibt, ist alles in Ordnung.

Am Ende sollte jede Zeile zeigen:

```
linguistic: 14 / 14 Subjects
glove: 14 / 14 Subjects
zrep_magnitudes: 14 / 14 Subjects
all_layer_activations: 14 / 14 Subjects
all_layer_z_representations: 14 / 14 Subjects
```

Die ungleiche Anzahl ist erwartbar, weil die fünf Bedingungen innerhalb jedes Subjects nacheinander gerechnet werden. Die All-Layer-Modelle brauchen wegen ihrer 9.216 Ausgangsfeatures beziehungsweise 36.864 verzögerten Features voraussichtlich am längsten.

ok. dann muss ich jetzt einfach warten bis alles gelaufen ist.

andere frage: Angenommen das skript ist fertig gelaufen - wie geht es weiter um figure 2 zu reproduzieren?

Dauer: 22 Sek. >

Genau. Nach Abschluss der 70 Encoding-Runs fehlen noch zwei zentrale Bausteine: die ISC-Noise-Ceiling und die Zuordnung der 1000 Parcels zu den zehn linken Sprach-ROIs.

Der vollständige Weg ist:

1. Encoding-Ergebnisse kontrollieren

Alle fünf Ordner müssen jeweils 14 Scoredateien enthalten:

```
Nur Text

results/pieman_encoding_linguistic/
results/pieman_encoding_glove/
results/pieman_encoding_zrep_magnitudes/
results/pieman_encoding_all_layer_activations/
results/pieman_encoding_all_layer_z_representations/
```

Gesamt:

```
Nur Text

5 Bedingungen × 14 Subjects = 70 Scoredateien
```

2. ISC-Noise-Ceiling berechnen

Für jedes Subject benötigen wir eine Datei mit 1000 parcelweisen ISC-Werten:

```
Nur Text

pieman_data/isc/sub-001.npy
...
pieman_data/isc/sub-016.npy
```

Jede Datei sollte die Form `(1000,)` haben. Vereinfacht wird dabei die Zeitreihe jedes Subjects parcelweise mit der mittleren Zeitreihe der übrigen 13 Subjects korreliert.

Dafür müssen wir als Nächstes das ISC-Skript des Repos prüfen. Nach Ende des Jobs suche es mit:

```
Bash

cd /nethome/phussein/brains/bert-brains

find . -type f -iname "*isc*.py" -print
```

Falls du `compute_isc.py` findest, lädst du es hier hoch.

3. Parcel-zu-ROI-Zuordnung prüfen

Das Originalnotebook benötigt:

```
Nur Text

Parcel2ROI_new.npy
```

Suche nach vorhandenen Zuordnungen:

```
Bash

find . -type f \( \
  -iname "*Parcel2ROI*.npy" \
  -o -iname "*parcel*roi*.npy" \
\) -print
```

Wir benötigen eine Zuordnung für dieselben 1000 Schaefer-Parcels wie im Encoding. Relevant sind diese zehn linken ROIs:

```
Nur Text

HG
PostTemp
AntTemp
AngG
IFG
IFGorb
MFG
vmPFC
dmPFC
PMC
```

4. Encoding-Ergebnisse richtig laden

Deine Dateien haben die Form:

```
Nur Text

(1000, 4)
```

Bei deiner angepassten Pieman-Regression befindet sich der tatsächlich berechnete Score in Spalte `0`:

```python
scores = np.load(path)[:, 0]
```

Das Originalnotebook verwendet dagegen:

```python
scores = np.load(path)[:, 3]
```

Das darf für Pieman nicht unverändert übernommen werden. Die Spalten `1–3` bleiben in deinem Skript null, weil Pieman nur eine Featuregruppe verwendet.

5. Auf Prozent der Noise Ceiling normalisieren

Das Originalnotebook transformiert zunächst Korrelationen mit Fisher-z:

```python
encoding_z = np.arctanh(encoding_correlations)
isc_z = np.arctanh(isc_correlations)
```

Dann werden die Parcels innerhalb eines ROI gemittelt:

```python
roi_encoding = encoding_z[roi_parcels].mean()
roi_isc = isc_z[roi_parcels].mean()
```

Anschließend:

```python
percent_noise_ceiling = 100 * roi_encoding / roi_isc
```

Werte über 100 werden wie im Original auf 100 begrenzt.

Die resultierende Struktur ist:

```
5 Modelle × 14 Subjects × 10 ROIs
```

6. Figure 2 zeichnen

Für jede Kombination aus Modell und ROI zeigt der Plot:

- Median über die 14 Subjects
- 95%-Bootstrap-Konfidenzintervall
- 10.000 Bootstrap-Samples
- y-Achse: Prozent der ISC-Noise-Ceiling

Die fünf Linien beziehungsweise Markergruppen sind:

```
Nur Text

Linguistic Features
GLoVe Embeddings
Transformation Magnitude
Transformer Embeddings
Transformations
```

Das vorhandene `Results_Figures.ipynb` enthält den ursprünglichen Plotcode, muss aber deutlich angepasst werden:

- nur `pieman` statt `black` und `slumlordreach`
- deine 14 Subject-IDs
- deine fünf Ergebnisordner
- lokale ISC-Pfade
- lokale ROI-Zuordnung
- Score-Spalte `[:, 0]` statt `[:, 3]`
- nur die zehn linken Sprach-ROIs

Der unmittelbar nächste Schritt nach dem Encoding-Job ist daher: `compute_isc.py` und `Parcel2ROI_new.npy` suchen beziehungsweise hochladen. Erst danach können wir das angepasste Figure-2-Auswertungsskript zuverlässig fertigstellen.

kann ich parcel to ROI und noise ceiling jetzt schon machen - wo das encoding noch nicht durchgelaufen ist?

Ja, beides ist vollständig unabhängig von den laufenden Encoding-Modellen. Du kannst es parallel vorbereiten beziehungsweise berechnen.

Die Datenabhängigkeiten sind:

Die laufenden Encoding-Dateien werden dadurch nicht verändert.

Als Nächstes suche beide Bestandteile:

```bash
cd /nethome/phussein/brains/bert-brains

find . -type f \( \
  -iname "*isc*.py" \
  -o -iname "*Parcel2ROI*.npy" \
  -o -iname "*parcel*roi*.npy" \
\) -print
```

Zusätzlich vorhandene ISC-Verzeichnisse prüfen:

```bash
find . -type d -iname "*isc*" -print
```

Wichtig: Starte noch kein unbekanntes ISC-Skript blind, da es möglicherweise fest kodierte Pfade und Subjectlisten für `black` oder `slumlordreach` enthält.

Du kannst aber gefahrlos:

- nach den Skripten und Dateien suchen,
- deren Inhalt anzeigen oder hier hochladen,
- die Pieman-Anpassung vorbereiten,
- anschließend einen separaten CPU-Condor-Job für ISC einreichen.

Der ISC-Job liest dieselben fMRI-Dateien, schreibt aber nur in einen separaten Ordner wie:

```
Nur Text

pieman_data/isc/
```

Er kollidiert daher nicht mit dem laufenden Encoding-Job. Je nach Cluster kann paralleles Lesen derselben NIfTI-Dateien etwas zusätzliche I/O erzeugen, funktional ist das aber unproblematisch.

```
(base) [phussein@submit ~]$ cd /nethome/phussein/brains/bert-brains

find . -type f \( \
  -iname "*isc*.py" \
  -o -iname "*Parcel2ROI*.npy" \
  -o -iname "*parcel*roi*.npy" \
\) -print
./analysis/compute_isc.py
./analysis/run_isc_jobs.py
(base) [phussein@submit bert-brains]$ find . -type d -iname "*isc*" -print
(base) [phussein@submit bert-brains]$
```

Gut: Die ISC-Skripte sind vorhanden, aber eine fertige `Parcel2ROI`-Datei und ein ISC-Ausgabeordner fehlen aktuell.

Als Nächstes brauche ich diese beiden Dateien:

```
Nur Text

analysis/compute_isc.py
analysis/run_isc_jobs.py
```

Lade sie bitte hier hoch. Dann kann ich prüfen:

- welche Eingabedateien `compute_isc.py` erwartet,
- ob es bereits parcelweise Daten voraussetzt,
- welche Subjectliste verwendet wird,
- wie Leave-one-subject-out-ISC berechnet wird,
- welche Anpassungen für Pieman notwendig sind,
- wie wir daraus einen separaten Condor-Job machen.

Suche parallel nach Code, der `Parcel2ROI` erzeugt oder verwendet:

```bash
cd /nethome/phussein/brains/bert-brains

rg -n \
  "Parcel2ROI|parcel.*ROI|ROI.*parcel" \
  --glob "*.py" \
  --glob "*.ipynb" \
  --glob "*.md"
```

Falls `rg` auf dem Cluster fehlt:

```bash
grep -RniE \
  "Parcel2ROI|parcel.*ROI|ROI.*parcel" \
  . \
  --include="*.py" \
  --include="*.md"
```

Außerdem nach ROI-Atlanten oder Masken suchen:

```bash
find . -type f \( \
  -iname "*roi*" \
  -o -iname "*language*mask*" \
  -o -iname "*schaefer*" \
\) -print
```

Die ISC-Anpassung können wir sofort vorbereiten. Bei `Parcel2ROI` müssen wir zunächst feststellen, ob das Repo die Datei generiert oder ob sie zu den ursprünglichen, nicht mit Git ausgelieferten Daten gehört.

(base) [phussein@submit analysis]$ ls
banded_ridge_regression.py
run_encoding_models_banded.py run_jupyter.sh
calculate_bootstrap_pvalue.py
run_encoding_models_headwise_banded.py run_pvalue_jobs.py
compute_isc.py run_encoding_pieman.py
run_save_jobs.py
headwise_banded_ridge_regression.py

run_encoding_pieman2.py save_mean_volumes.py
joblist_pieman.txt run_isc_jobs.py
(base) [phussein@submit analysis]$

Die beiden relevanten Dateien sind da:

```
Nur Text

analysis/compute_isc.py
analysis/run_isc_jobs.py
```

Lade bitte beide hier hoch. Die übrigen Dateien benötige ich für diesen Schritt noch nicht.

Falls Hochladen gerade umständlich ist, kannst du alternativ den Inhalt ausgeben:

```bash
cd /nethome/phussein/brains/bert-brains

sed -n '1,320p' analysis/compute_isc.py
sed -n '1,320p' analysis/run_isc_jobs.py
```

Für `Parcel2ROI` brauchen wir zusätzlich noch die Ausgabe von:

```bash
find . -type f \( \
  -iname "*roi*" \
  -o -iname "*language*mask*" \
  -o -iname "*schaefer*" \
\) -print
```

Dann behandeln wir die beiden Aufgaben getrennt:

1. `compute_isc.py` für deine 14 Pieman-Subjects anpassen und als Condor-Job starten.
2. Ermitteln, wie die 1000 Schaefer-Parcels den zehn linken Sprach-ROIs zugeordnet werden.

compute_isc.py
Code

run_isc_jobs.py
Code

Dauer: 27 Sek. >

Die Originalskripte können für Pieman nicht unverändert ausgeführt werden.

`compute_isc.py` erwartet:

```
Nur Text

<dataset>_data/sub-XXX_parcelwise_data.npy
<dataset>_data/nuisance_features.npy
<dataset>_data/isc/
```

Bei dir fehlen die `sub-XXX_parcelwise_data.npy`-Dateien. Außerdem sind Pfad und Subjectlisten fest auf `black` und `slumlordreach` eingestellt.

Wichtig ist auch diese Zeile:

```
Python

heldout_average = np.mean(full_data, axis=0)
```

Trotz des Namens `heldout_average` wird das jeweilige Ziel-Subject dabei nicht ausgeschlossen. Für eine möglichst genaue Code-Reproduktion sollten wir dieses Verhalten zunächst beibehalten. Bei nur 14 Subjects kann die Selbsteinbeziehung die Noise Ceiling allerdings stärker beeinflussen; später wäre eine Leave-one-subject-out-Kontrollanalyse sinnvoll.

Bevor ich das Pieman-Skript erstelle, brauche ich noch:

```
Nur Text

analysis/save_mean_volumes.py
```

Diese Datei dürfte die erwarteten `sub-XXX_parcelwise_data.npy`-Zeitreihen erzeugen. Bitte lade sie hoch oder zeige sie an:

```
Bash

cd /nethome/phussein/brains/bert-brains
sed -n '1,360p' analysis/save_mean_volumes.py
```

`run_isc_jobs.py` benötigen wir wahrscheinlich nicht weiter: Es schreibt lediglich eine `joblist.txt` für die beiden Originaldatensätze. Für Pieman erstellen wir einen eigenen Condor-Job für deine 14 Subjects.

Die geplante ISC-Pipeline ist dann:

1. NIfTI-Daten mit `Schaefer1000_3mm.nii.gz` parzellieren.
2. Dieselben Zeitpunkte und Rand-TRs wie beim Encoding verwenden.
3. Für jedes Subject eine Matrix `1000 Parcels × Zeitpunkte` erzeugen.
4. Parcelweise ISC in drei zeitlichen Folds berechnen.
5. Pro Subject eine `(1000,)`-Datei unter `pieman_data/isc/` speichern.

Das kann parallel zum laufenden Encoding-Job passieren.

kann es sein dass ich ein paar subjects auf slumlord und pieman habe? würde das helfen?

Dauer: 8 Sek.

Ja, einzelne Subject-IDs können in mehreren Narratives-Stories vorkommen. Das hilft aber nur begrenzt.

Für die ISC-Noise-Ceiling müssen alle verglichenen Personen dieselbe Geschichte gehört haben. Daher darfst du nicht:

```
Nur Text

Pieman-Zeitreihe von sub-016
```

mit:

```
Nur Text

Slumlordreach-Zeitreihen anderer Subjects
```

korrelieren. Unterschiedliche Geschichten haben andere Inhalte, Zeitachsen und TR-Anzahlen. Die Pieman-ISC muss ausschließlich aus den 14 Pieman-Aufnahmen berechnet werden.

Ein überlappendes Subject hilft in diesen Bereichen:

- Dieselbe anatomische beziehungsweise Schaefer-Parzellierung kann verwendet werden, sofern alle Daten im gleichen Standardraum liegen.
- Wenn du später auch `slumlordreach` vollständig analysierst, kann diese Person in beiden storyspezifischen Analysen vorkommen.
- Mehrere Stories derselben Person könnten für eine zusätzliche, innerhalb einer Person durchgeführte Generalisierungsanalyse interessant sein.

Für die Stichprobengröße darf dieselbe Person aber nicht einfach zweimal als unabhängiges

Subject gezählt werden. Wenn du Pieman und Slumlordreach später aggregierst, müsste die Abhängigkeit berücksichtigt oder pro Person zunächst über Stories gemittelt werden.

Für unseren unmittelbaren Figure-2-Nachbau ändert es daher nichts:

```
Nur Text

Encoding: 14 Pieman-Subjects
ISC: dieselben 14 Pieman-Subjects
```

Am `Parcel2ROI`-Problem könnte vorhandenes Slumlordreach-Material allerdings helfen: Die Zuordnung ist atlasbezogen, nicht storybezogen. Falls irgendwo außerhalb des Repos eine Slumlordreach-Datei wie

```
Nur Text

Parcel2ROI_new.npy
```

liegt und exakt denselben `Schaefer1000_3mm.nii.gz`-Atlas verwendet, könnten wir sie auch für Pieman übernehmen. Die ISC-Dateien von Slumlordreach können wir dagegen nicht wiederverwenden.

wie kann ich im repo nach parcel2roi suchen?

Im gesamten Repo suchst du so:

```bash
cd /nethome/phussein/brains/bert-brains

find . -type f -iname "*parcel*2*roi*" -print
```

Da die Schreibweise variieren kann, zusätzlich breiter suchen:

```bash
find . -type f \( \
  -iname "*parcel*roi*" \
  -o -iname "*parcel*network*" \
  -o -iname "*roi*mapping*" \
  -o -iname "*language*roi*" \
\) -print
```

Auch im Code nach Erwähnungen suchen:

```bash
rg -ni \
  "Parcel2ROI|parcel.?to.?roi|parcel.*roi|roi.*parcel" \
  . \
  --glob "*.py" \
  --glob "*.ipynb" \
  --glob "*.md"
```

Falls `rg` nicht installiert ist:

```bash
grep -RniE \
  "Parcel2ROI|parcel.?to.?roi|parcel.*roi|roi.*parcel" \
  . \
  --include="*.py" \
  --include="*.md"
```

Weil die Originalpfade im Notebook außerhalb des Git-Repos liegen, suche anschließend in deinem gesamten Projektverzeichnis:

```bash
find /nethome/phussein/brains -type f \( \
  -iname "*Parcel2ROI*.npy" \
  -o -iname "*parcel*roi*.npy" \
\) -print
```

Falls dort nichts auftaucht, noch im Scratch-Bereich:

```bash
find /scratch/phussein -type f \( \
  -iname "*Parcel2ROI*.npy" \
  -o -iname "*parcel*roi*.npy" \
\) -print 2>/dev/null
```

Die relevantesten Treffer wären:

```
Nur Text
```

```
Parcel2ROI.npy
Parcel2ROI_new.npy
```

Zeige mir anschließend die Ausgabe. Entscheidend ist dann, ob die Datei 1000 Einträge enthält und zu deinem `Schaefer1000_3mm.nii.gz` gehört.

```
-bash: rg: command not found
(base) [phussein@submit bert-brains]$ grep -RniE \
  "Parcel2ROI|parcel.?to.?roi|parcel.*roi|roi.*parcel" \
  . \
  --include="*.py" \
  --include="*.md"
./analysis/
banded_ridge_regression.py:562:#roi_mapping=np.load('/
jukebox/griffiths/bert-brains/black_data/Parcel2ROI.npy')
```

Das bestätigt: Im Repo wird `Parcel2ROI` nur erwähnt, aber die Datei selbst wird offenbar nicht mitgeliefert. Sie lag bei den Autor:innen außerhalb des Git-Repos unter:

Nur Text

```
/jukebox/griffiths/bert-brains/black_data/Parcel2ROI.npy
```

Suche jetzt in deinen eigenen Projekt- und Scratch-Verzeichnissen nach der tatsächlichen Datei:

Bash

```
find /nethome/phussein/brains /scratch/phussein \
  -type f \( \
    -iname "*Parcel2ROI*.npy" \
    -o -iname "*parcel*roi*.npy" \
  \) -print 2>/dev/null
```

Suche zusätzlich nach möglichen ROI-NIfTI-Dateien:

Bash

```
find /nethome/phussein/brains /scratch/phussein \
  -type f \( \
    -iname "*language*roi*" \
    -o -iname "*language*mask*" \
    -o -iname "*roi*.nii" \
    -o -iname "*roi*.nii.gz" \
```

```
\) -print 2>/dev/null
```

Wenn beide Suchen leer bleiben, müssen wir `Parcel2ROI_new.npy` selbst rekonstruieren. Dafür benötigen wir:

1. deinen Atlas:

```
Nur Text

pieman_data/Schaefer1000_3mm.nii.gz
```

2. die Masken beziehungsweise Definitionen der zehn Sprach-ROIs:

```
Nur Text

HG, PostTemp, AntTemp, AngG, IFG,
IFGorb, MFG, vmPFC, dmPFC, PMC
```

Der Schaefer-Atlas allein reicht dafür nicht aus: Diese zehn Kategorien entsprechen nicht einfach den üblichen sieben oder siebzehn Schaefer-Netzwerken. Wir brauchen die konkrete ROI-Definition der Studie oder die originale Mapping-Datei.

Wenn keine lokale Datei existiert, wären die nächsten sinnvollen Optionen:

- im Original-Dataset beziehungsweise DataLad-Inhalt nachsehen;
- das GitHub-Repository und zugehörige veröffentlichte Datenarchiv prüfen;
- die Autor:innen nach `Parcel2ROI_new.npy` fragen;
- alternativ eine neue, klar dokumentierte ROI-Zuordnung aus einem verfügbaren Sprach-ROI-Atlas erstellen, was dann aber keine exakte Reproduktion wäre.

Führe zuerst die beiden `find`-Befehle aus. Parallel können wir die ISC-Pipeline weiterbauen; sie benötigt `Parcel2ROI` noch nicht, sondern nur den Schaefer-Atlas.

gibt es im paper hinweise wie parcel2ROI zu stande kommt?

Dauer: 29 Sek. >

Ja. Das Paper beschreibt ziemlich genau, wie die Zuordnung grundsätzlich erstellt wurde, aber nicht vollständig genug, um alle zehn ROIs exakt ohne Zusatzdateien zu rekonstruieren.

Laut Methods wurde folgendermaßen vorgegangen:

ROI-Gruppe	Quelle	Zuordnung zu Schaefer-Parcels
PostTemp, AntTemp, AngG, IFG, IFGorb, MFG	Fedorenko Functional Language Localizer	Parcel gehört zum ROI, wenn mehr als 50 % seiner Voxel mit der ROI-Maske im MNI-Raum überlappen
HG	Harvard-Oxford Atlas	Überlappung mit Heschl's Gyrus
vmPFC, dmPFC, PMC	Schaefer-Parzellierung	Von den Autor:innen manuell definiert

Die entscheidende Passage lautet sinngemäß:

> Each ROI was defined as parcels from the 1000-parcel atlas for which over 50% of parcel voxels overlapped the ROI in MNI space.

Die Ausgangsdaten waren:

- Schaefer-Atlas mit 1000 cortical parcels
- fMRI-Daten im Raum `MNI152NLin2009cAsym`
- Fedorenko-Sprachmasken von EvLab Functional Localizer
- Harvard-Oxford Atlas für HG
- manuell ausgewählte Schaefer-Parcels für vmPFC, dmPFC und PMC

Damit dürfte `Parcel2ROI_new.npy` ungefähr so erzeugt worden sein:

```
Nur Text

Für jedes der 1000 Schaefer-Parcels:
    Überlappung mit jeder ROI-Maske berechnen
    wenn > 50 % der Parcel-Voxel in einer ROI-Maske liegen:
        Parcel dieser ROI zuweisen
```

Das Problem: Für `vmPFC`, `dmPFC` und `PMC` nennt das Paper keine konkreten Schaefer-Parcelnummern. Deshalb können wir diese drei ROIs anhand des Papers allein nicht exakt reproduzieren.

Die gute Nachricht ist: Im Paper steht auch, dass „Source data" bereitgestellt werden. Darin könnten Parcelnamen oder ROI-Zuordnungen enthalten sein. Der nächste sinnvolle Schritt wäre daher:

1. Source-Data-Dateien des Papers untersuchen.
2. Nach Parcelnummern beziehungsweise ROI-Labels suchen.
3. Falls dort nichts enthalten ist, Supplement und Repository-Historie prüfen.

4. Erst als letzte Option das Mapping selbst aus den veröffentlichten Masken rekonstruieren.

Die Beschreibung steht im Methods-Abschnitt „MRI data preprocessing and cortical parcellation“ des Nature-Communications-Papers. Für sechs Sprach-ROIs und HG ist das Verfahren reproduzierbar; für die drei manuell definierten höheren ROIs benötigen wir zusätzliche Informationen.

ok. habe die emailadressen. schreib mir bitte eine email vpr

Dauer: 12 Sek. >

E-Mail

E-Mail verbinden

Empfänger:in

Request for Parcel-to-ROI mapping used in Kumar et al. (2024)

Dear Dr. Kumar,

I am currently reproducing the analyses from your paper “Shared functional specialization in transformer-based language models and the human brain” as part of a university coding project.

I am working on reproducing Figure 2 using the Pieman dataset and the Schaefer 1000-parcel atlas. The paper describes how the parcels were assigned to the ten language ROIs, but I could not find the corresponding `Parcel2ROI.npy` or `Parcel2ROI_new.npy` file in the GitHub repository.

Would it be possible to share the parcel-to-ROI mapping used in the paper, or alternatively the lists of Schaefer parcel IDs assigned to each ROI? In particular, the manually defined vmPFC, dmPFC, and PMC assignments would be very helpful.

Thank you very much for your time and help!

Best regards,
Paulina Hussein

etwas kürzer

E-Mail

E-Mail verbinden

Empfänger:in

Parcel-to-ROI mapping for Figure 2

Dear Dr. Kumar,

I am reproducing Figure 2 from your paper using the Pieman dataset. Could you please share the `Parcel2ROI.npy/Parcel2ROI_new.npy` mapping used in the analysis, or the corresponding Schaefer parcel IDs for each ROI?

I could not find this file in the GitHub repository, and it would be very helpful for my reproduction.

Thank you!

Best regards,
Paulina Hussein

schreib noch dazu, dass generell die jukebox dinge nicht mitgefliefert sind/frag wo man die findet

E-Mail

E-Mail verbinden

Empfänger:in

Missing data files for Figure 2 reproduction

Dear Dr. Kumar,

I am reproducing Figure 2 from your paper using the Pieman dataset. Could you please share the `Parcel2ROI.npy/Parcel2ROI_new.npy` mapping used in the analysis, or the corresponding Schaefer parcel IDs for each ROI?

I also noticed that several files referenced through the `/jukebox/griffiths/bert-brains/` paths are not included in the GitHub repository. Are these files publicly available elsewhere?

Thank you very much!

Best regards,
Paulina Hussein

Dear Dr. Kumar, dear Dr. Sumers,

I am currently reproducing from your paper "Shared functional specialization in transformer-based language models and the human brain" using the Pieman dataset.
The reproduction of Figure 2 requires the `Parcel2ROI.npy/ Parcel2ROI_new.npy` mapping used in the analysis, or the corresponding Schaefer parcel IDs for each ROI. Would you please share those?

I also noticed that several files referenced through the `/ jukebox/griffiths/bert-brains/` paths are not included in the GitHub repository. Are these files publicly available elsewhere?

Thank you very much!

Best regards,
Paulina Hussein

bitte an den anfang etwas ehr motivation

E-Mail

E-Mail verbinden

Empfänger:in

Files needed for Figure 2 reproduction

Dear Dr. Kumar, dear Dr. Sumers,

I found your work on the relationship between transformer representations and the brain very interesting and am currently reproducing parts of your paper “Shared functional specialization in transformer-based language models and the human brain” for a university project, using the Pieman dataset.

Reproducing Figure 2 requires the `Parcel2ROI.npy/Parcel2ROI_new.npy` mapping or the corresponding Schaefer parcel IDs for each ROI. Would you be able to share these?

I also noticed that several files referenced through the `/jukebox/griffiths/bert-brains/` paths are not included in the GitHub repository. Are they publicly available elsewhere?

Thank you very much!

Best regards,
Paulina Hussein

ja ok, abgeschickt. was mach ich jetzt?

Dauer: 12 Sek. >

Jetzt machen wir mit der ISC-Noise-Ceiling weiter; dafür brauchen wir die Antwort der Autor:innen nicht.

Der nächste konkrete Schritt ist: Lade mir bitte diese Datei hoch:

```
Nur Text

analysis/save_mean_volumes.py
```

Sie dürfte zeigen, wie das Original-Repo die fMRI-Daten in die benötigten parcelweisen Zeitreihen umwandelt:

```
Nur Text

sub-XXX_parcelwise_data.npy
```

Alternativ kannst du sie anzeigen:

```
Bash

cd /nethome/phussein/brains/bert-brains
sed -n '1,360p' analysis/save_mean_volumes.py
```

Danach erstelle ich dir ein Pieman-ISC-Skript, das:

- deine 14 NIfTI-Dateien lädt,
- denselben Schaefer-1000-Atlas verwendet,
- dieselben TRs wie das Encoding auswählt,
- die 1000 parcelweisen Zeitreihen berechnet,
- die ISC in drei zeitlichen Folds berechnet,
- 14 Dateien unter `pieman_data/isc/` speichert.

Parallel lässt du den laufenden Encoding-Job einfach weiterlaufen. Die ROI-Zuordnung bearbeiten

wir, sobald die Autor:innen antworten oder wir sie in den Source Data finden.

save_mean_volum...
Code

Dauer: 1 Min. 36 Sek. >

`save_mean_volumes.py` war nicht für ISC gedacht. Ich habe deshalb ein eigenes Pieman-Skript erstellt, das die Vorverarbeitung deines Encoding-Skripts nachbildet:

Lade es ins Repo-Hauptverzeichnis:

Nur Text

```
/nethome/phussein/brains/bert-brains/compute_isc_pieman.py
```

Setze anschließend `brains.sh` auf:

Bash

```
#!/bin/bash
set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh
conda activate fmri_encoding

cd /nethome/phussein/brains/bert-brains

echo "Starting Pieman ISC computation"
python compute_isc_pieman.py
echo "Finished successfully"
```

Für `brains.sub`:

Nur Text

```
request_CPUs   = 2
request_memory = 16G
```

GPU-Zeilen entfernen oder auskommentieren. Dann:

Bash

```
chmod +x /nethome/phussein/brains/brains.sh
```

```
cd /nethome/phussein/brains
condor_submit brains.sub
```

Das Skript:

- verarbeitet alle 14 Subjects,
- erzeugt zuerst `sub-XXX_parcelwise_data.npy`,
- verwendet dieselben 282 anfänglichen und anschließend 262 verbleibenden TRs wie das Encoding,
- berechnet ISC in drei zusammenhängenden Zeit-Folds,
- bezieht das Ziel-Subject wie im Paper absichtlich in den Gruppendurchschnitt ein,
- speichert 14 Arrays mit jeweils `(1000,)` unter:

Nur Text

```
pieman_data/isc/
```

Dieser Job kann parallel zum Encoding-Job laufen. Die beiden schreiben in unterschiedliche Dateien.

compute_isc_pieman.py
Code

warum ist der job in der IDLE?

#!/bin/bash

set -e

source /nethome/phussein/miniconda3/etc/profile.d/conda.sh

conda activate fmri_encoding

cd /nethome/phussein/brains/bert-brains

echo "Starting Pieman ISC computation"

python compute_isc_pieman.py

echo "Finished successfully"

Das liegt sehr wahrscheinlich nicht an `brains.sh`. Ein Job bleibt `IDLE`, wenn Condor anhand von `brains.sub` noch keinen passenden Rechner findet.

Bei deiner bisherigen `brains.sub` waren diese sehr starken Anforderungen enthalten:

```
Nur Text
request_memory = 64G
request_GPUs = 1
requirements = (GPUs_Capability >= 8.0) && (GPUs_GlobalMemoryMb > 60000)
```

Für ISC brauchst du keine GPU. Prüfe die tatsächlich eingereichte Konfiguration:

```
Bash
cat /nethome/phussein/brains/brains.sub
```

Sie sollte ungefähr so aussehen:

```
Nur Text
```

```
initialdir = /nethome/phussein/brains
executable = brains.sh

output = /scratch/phussein/logs/brains/brains.sh.$(ClusterId).out
error  = /scratch/phussein/logs/brains/brains.sh.$(ClusterId).err
log    = /scratch/phussein/logs/brains/brains.sh.$(ClusterId).log

request_CPUs   = 2
request_memory = 16G

getenv = True

queue 1
```

Diese Zeilen dürfen nicht mehr aktiv sein:

```
Nur Text

request_GPUs = 1
requirements = ...
```

Den konkreten Idle-Grund zeigt Condor mit:

```
Bash

condor_q
```

Job-ID merken und dann:

```
Bash

condor_q -better-analyze <JOB-ID>
```

Falls du den Job bereits mit der alten `.sub`-Datei eingereicht hast, wirken nachträgliche Änderungen nicht auf den laufenden Job. Dann den Idle-Job entfernen:

```
Bash

condor_rm <JOB-ID>
```

und neu einreichen:

```
Bash
```

```
cd /nethome/phussein/brains
condor_submit brains.sub
```

Dein gezeigtes `brains.sh` selbst ist korrekt. Entscheidend für den `IDLE`-Status ist jetzt die Ausgabe von `condor_q -better-analyze <JOB-ID>`.

```
##################################################
####################
# HTCondor submit description file
# Everything with a leading # is a comment

initialdir              = /nethome/phussein/brains
executable              = brains.sh
output                  = /scratch/phussein/logs/brains/brains.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).out
error                   = /scratch/phussein/logs/brains/brains.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).err
log                     = /scratch/phussein/logs/brains/brains.sh.
$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).log
request_CPUs            = 1
request_memory          = 64G
request_GPUs            = 1
requirements            = (GPUs_Capability
>=8.0)&&(GPUs_GlobalMemoryMb > 60000)
#environment = WANDB_API_KEY=46736878....
getenv = True
queue 1
##################################################
####################
```

Das erklärt den `IDLE`-Status. Der Job fordert aktuell:

- eine GPU,
- mindestens 60 GB GPU-Speicher,
- GPU Compute Capability ≥ 8.0,
- zusätzlich 64 GB RAM.

Damit wartet Condor auf einen seltenen großen GPU-Knoten. Für die ISC-Berechnung ist das unnötig.

Ändere `brains.sub` zu:

```
Nur Text

######################################################################
# HTCondor submit file for Pieman ISC

initialdir = /nethome/phussein/brains
executable = brains.sh

output = /scratch/phussein/logs/brains/brains.sh.$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).ou
error  = /scratch/phussein/logs/brains/brains.sh.$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).er
log    = /scratch/phussein/logs/brains/brains.sh.$(ClusterId).$(Year)_$(Month)_$(Day)_$(SUBMIT_TIME).lo

request_CPUs   = 2
request_memory = 16G

getenv = True

queue 1
######################################################################
```

Entferne den bereits eingereichten Idle-Job:

```
Bash

condor_q
condor_rm <JOB-ID>
```

Dann mit der geänderten Datei neu einreichen:

```
Bash

cd /nethome/phussein/brains
condor_submit brains.sub
```

Danach sollte der Job deutlich schneller von `I` zu `R` wechseln:

```
Bash

condor_q
```

Falls er weiterhin länger idle bleibt:

```
Bash
```

```
condor_q -better-analyze <NEUE-JOB-ID>
```

Die GPU-Anforderung war hier der entscheidende Engpass.

immer noch in der idle

Dann brauchen wir den tatsächlichen Match-Grund. Die `.sub`-Datei allein reicht nicht, weil möglicherweise noch der alte Job läuft oder der Cluster zusätzliche Standardanforderungen ergänzt.

Zuerst:

```bash
condor_q
```

Nimm die neue Job-ID und führe aus:

```bash
condor_q -better-analyze <JOB-ID>
```

Prüfe außerdem, welche Ressourcen beim neuen Job tatsächlich gespeichert wurden:

```bash
condor_q <JOB-ID> -af \
ClusterId JobStatus RequestCpus RequestMemory RequestGpus Requirements
```

Achte besonders auf `RequestGpus`. Es sollte `0`, `undefined` oder leer sein – nicht `1`.

Mögliche Ursachen:

1. Es ist noch der alte GPU-Job:

```bash
condor_rm <ALTE-JOB-ID>
```

2. Die geänderte Datei wurde nicht gespeichert oder eine andere Datei eingereicht:

```bash
cd /nethome/phussein/brains
grep -nE "request_|requirements" brains.sub
```

Erwartet:

```
request_CPUs   = 2
request_memory = 16G
```

3. Dein bereits laufender Encoding-Job blockiert aufgrund eines nutzerspezifischen Joblimits einen zweiten Job. Dann meldet `better-analyze` typischerweise eine Policy- oder Limit-Ursache. In diesem Fall kann der ISC-Job erst starten, wenn der Encoding-Job fertig ist.
4. 16 GB sind gerade nicht verfügbar. Dann könntest du auf 8 GB reduzieren:

```
request_CPUs   = 1
request_memory = 8G
```

Reduziere die Ressourcen aber erst, nachdem wir die Analyse gesehen haben.

Poste bitte die komplette Ausgabe von:

```bash
condor_q -better-analyze <JOB-ID>
```

Dann können wir den konkreten Grund direkt ablesen.

(base) [phussein@submit brains]$ condor_q -better-analyze 112926.0

-- Schedd: submit.coli.uni-saarland.de : <134.96.104.7:9618?...
The Requirements expression for job 112926.000 is

    (((GPUs_Capability >= 8.000000000000000E+00) && (GPUs_GlobalMemoryMb > 60000)) && (TARGET.Arch ==

```
"X86_64") &&
      (TARGET.OpSys == "LINUX") && (TARGET.Disk >=
RequestDisk) && (TARGET.Memory >= RequestMemory) &&
      (TARGET.Cpus >= RequestCpus) &&
((TARGET.FileSystemDomain == MY.FileSystemDomain) ||
        (TARGET.HasFileTransfer))) && (Target.IsHighEndNode isnt
true)

Job 112926.000 defines the following attributes:

    DiskUsage = 2
    FileSystemDomain = "coli.uni-saarland.de"
    RequestCpus = 2
    RequestDisk = undefined (kb)
    RequestMemory = 16384 (mb)

The Requirements expression for job 112926.000 reduces to
these conditions:

         Slots
Step    Matched  Condition
-----  --------  ---------
[1]          37  GPUs_GlobalMemoryMb > 60000
[11]         37  TARGET.Cpus >= RequestCpus
[12]         28  [1] && [11]
[13]         47  TARGET.FileSystemDomain ==
MY.FileSystemDomain
[17]          8  Target.IsHighEndNode isnt true
[18]          0  [12] && [17]

No successful match recorded.
Last failed match: Thu Sep  3 17:28:15 2026

Reason for last match failure: no match found

112926.000:  Run analysis summary ignoring user priority.  Of 8
machines,
      8 are rejected by your job's requirements
      0 reject your job because of their own requirements
```

0 match and are already running your jobs
0 match but are serving other users
0 are able to run your job

WARNING: Be advised:
No machines matched the jobs's constraints

Die Ausgabe zeigt eindeutig: Die GPU-`requirements` sind beim Job weiterhin aktiv:

```
Nur Text

GPUs_Capability >= 8.0
GPUs_GlobalMemoryMb > 60000
```

`request_GPUs` wurde zwar entfernt, aber die `requirements`-Zeile war beim Absenden offenbar noch vorhanden. Änderungen an `brains.sub` ändern einen bereits eingereichten Job nicht nachträglich.

Entferne zuerst Job `112926.0`:

```
Bash

condor_rm 112926.0
```

Prüfe die gespeicherte Submit-Datei:

```
Bash

grep -nE "GPU|requirements|request_" \
/nethome/phussein/brains/brains.sub
```

Es sollten ausschließlich diese Ressourcen erscheinen:

```
Nur Text

request_CPUs   = 2
request_memory = 16G
```

Insbesondere darf keine aktive Zeile mit `requirements` vorkommen. Am sichersten sieht der Ressourcenabschnitt so aus:

```
Nur Text

request_CPUs   = 2
request_memory = 16G

# No GPU required
# request_GPUs = 1
# requirements = (GPUs_Capability >=8.0)&&(GPUs_GlobalMemoryMb > 60000)
```

Dann wirklich diese Datei neu einreichen:

```
Bash

cd /nethome/phussein/brains
condor_submit brains.sub
```

Direkt danach die neue Jobkonfiguration kontrollieren:

```
Bash

condor_q <NEUE-JOB-ID> -af \
ClusterId RequestCpus RequestMemory RequestGpus Requirements
```

In `Requirements` dürfen danach weder `GPUs_Capability` noch `GPUs_GlobalMemoryMb` stehen. Dann kann der Job auf einem normalen CPU-Knoten starten.

ok, der job läuft jetzt

Sehr gut. Jetzt laufen parallel:

- der Figure-2-Encoding-Job mit insgesamt 70 Runs;
- der ISC-Job für 14 Subjects × 1000 Parcels.

Den ISC-Fortschritt siehst du im neuen Output-Log:

```
Bash

tail -f /scratch/phussein/logs/brains/brains.sh.<ISC-JOB-ID>*.out
```

`Ctrl+C` beendet nur die Logansicht.

Das Skript sollte zunächst nacheinander Meldungen ausgeben wie:

```
Nur Text
```

```
Parcellating: sub-001
Saved: .../sub-001_parcelwise_data.npy (1000, 262)
```

Danach folgen die ISC-Dateien:

```
Nur Text

Saved ISC: sub-001 (1000,) ...
```

Den Zwischenstand kannst du zählen:

```bash
cd /nethome/phussein/brains/bert-brains

echo "Parcel data:"
find pieman_data -maxdepth 1 \
  -name "sub-*_parcelwise_data.npy" | wc -l

echo "ISC:"
find pieman_data/isc -maxdepth 1 \
  -name "sub-*.npy" 2>/dev/null | wc -l
```

Am Ende erwartest du:

```
Nur Text

Parcel data: 14
ISC: 14
```

Danach prüfen wir Shapes, Wertebereiche und die Übereinstimmung der 262 Zeitpunkte mit dem Encoding. Für den finalen Figure-2-Plot fehlt dann nur noch das `Parcel2ROI`-Mapping beziehungsweise die Antwort der Autor:innen.

was ist ISC ?

ISC bedeutet **Intersubject Correlation**, also die Korrelation der Hirnaktivität zwischen verschiedenen Versuchspersonen.

Für jedes Subject und jedes der 1000 Parcels wird gefragt:

> Wie ähnlich ist die fMRI-Zeitreihe dieser Person der durchschnittlichen Zeitreihe aller Personen, während sie dieselbe Geschichte hören?

Vereinfacht:

```
Nur Text

fMRI-Zeitreihe von sub-001 in Parcel 1
                    ↕ Korrelation
durchschnittliche Zeitreihe aller Subjects in Parcel 1
```

Das wird für alle Subjects und alle 1000 Parcels berechnet.

Warum brauchen wir ISC?

Die fMRI-Aktivität enthält sowohl:

- durch die Geschichte hervorgerufene, zwischen Personen ähnliche Aktivität;
- individuelles Rauschen und Messrauschen.

Wenn ein Parcel bei verschiedenen Menschen sehr ähnlich auf Pieman reagiert, hat es eine hohe ISC. Dann ist dort viel zuverlässig stimulusgetriebene Aktivität vorhanden. Ein Sprachmodell kann sinnvollerweise höchstens diesen reproduzierbaren Anteil vorhersagen.

Deshalb verwendet das Paper ISC als **Noise Ceiling**:

$$\text{Prozent der Noise Ceiling} = 100 \times \frac{\text{Encoding-Korrelation}}{\text{ISC}}$$

Beispiel:

```
Nur Text

Encoding-Korrelation = 0,10
ISC                  = 0,40
```

Dann erklärt das Modell:

$$100 \times \frac{0{,}10}{0{,}40} = 25\%$$

der geschätzten vorhersagbaren Aktivität.

Eine hohe ISC bedeutet nicht automatisch, dass das Sprachmodell gut ist. Sie bedeutet zunächst nur, dass die Reaktion in diesem Hirnbereich zwischen Personen zuverlässig ist. Figure 2 setzt die Modellleistung relativ zu dieser Zuverlässigkeit.